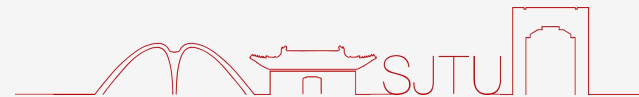




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Chapitre 5 : Méthode de Newton

孔思佳

2026年05月

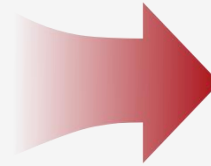
—— 饮水思源 · 爱国荣校 ——

Flou de bougé



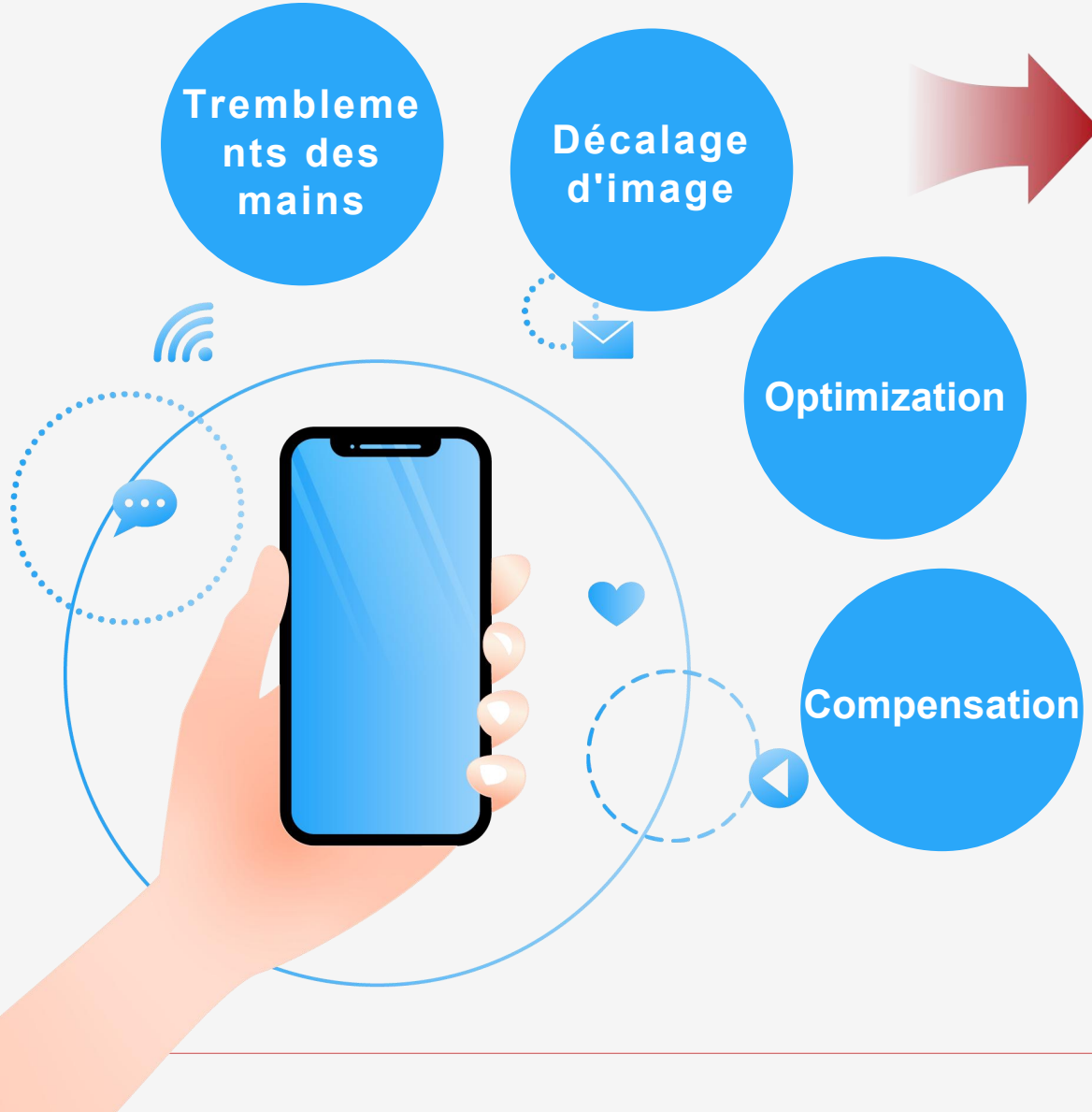
On rencontre souvent des problèmes de flou de bougé.

Traitement
par
optimisation



La stabilisation d'image devient une fonctionnalité très efficace. Cette fonctionnalité permet d'**optimiser la réduction du flou**.

Flou de bougé



Modèle mathématique

$$\min_x f(x) = \text{Flou de l'image } (x)$$

- (x) : Vecteur de compensation (translation + rotation)
- Exigences : **Efficace**, peu d'itérations

L'algorithme doit calculer comment compenser les tremblements de la main en un temps encore plus court. **Comment est-ce possible ?**

Compenser un temps encore plus court

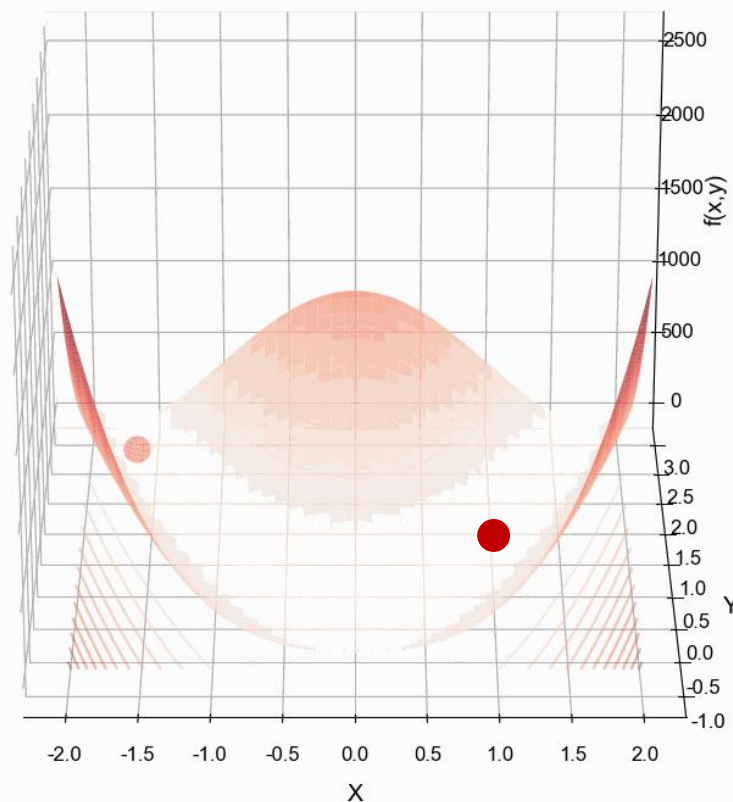


上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

梯度下降法 (Gradient Descent)

更新规则: $x_{k+1} = x_k - \alpha \nabla f(x_k)$

★ 全局最小值



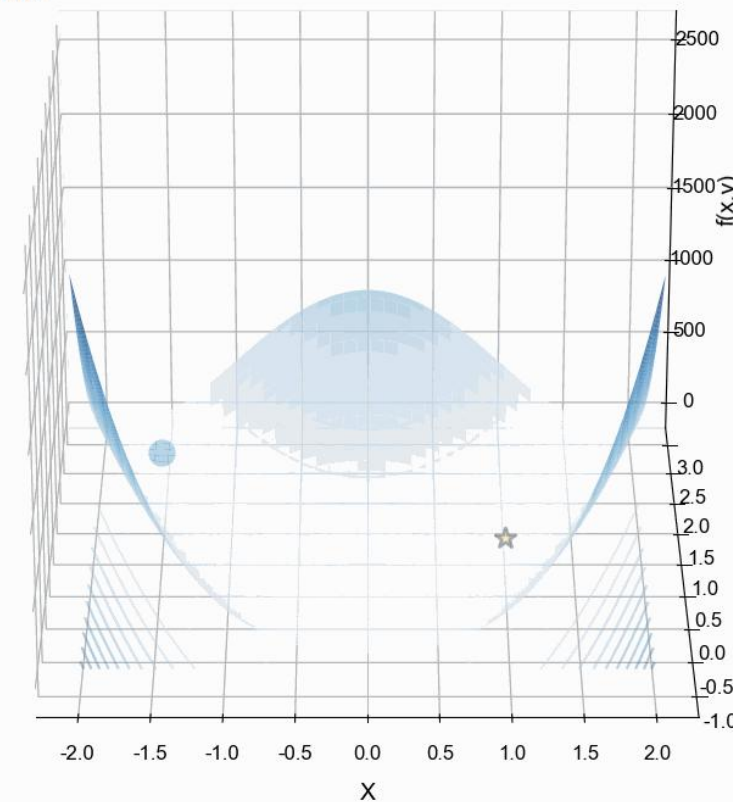
Descente de gradient : chaque étape se déplace dans la direction la plus raide, mais ne peut pas converger rapidement vers le point le plus bas.

牛顿法 (Newton's Method)

更新规则: $x_{k+1} = x_k - H^{-1} \nabla f(x_k)$

★ 全局最小值

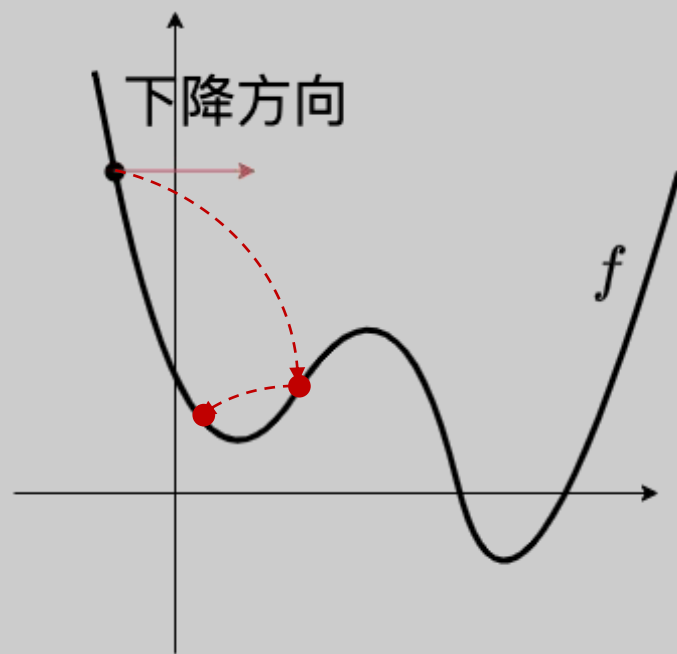
迭代 1
误差: 2.911406



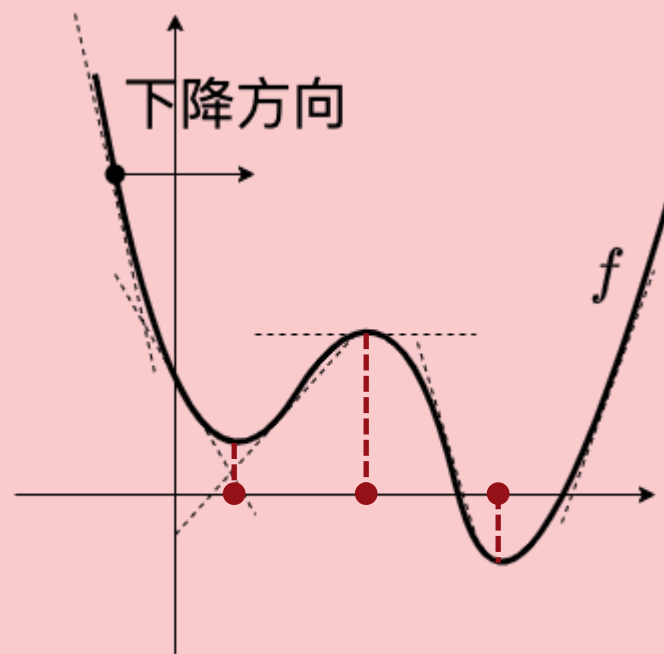
Méthode de Newton : Introduction d'une méthode du second ordre à convergence plus rapide

Développement limité

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$

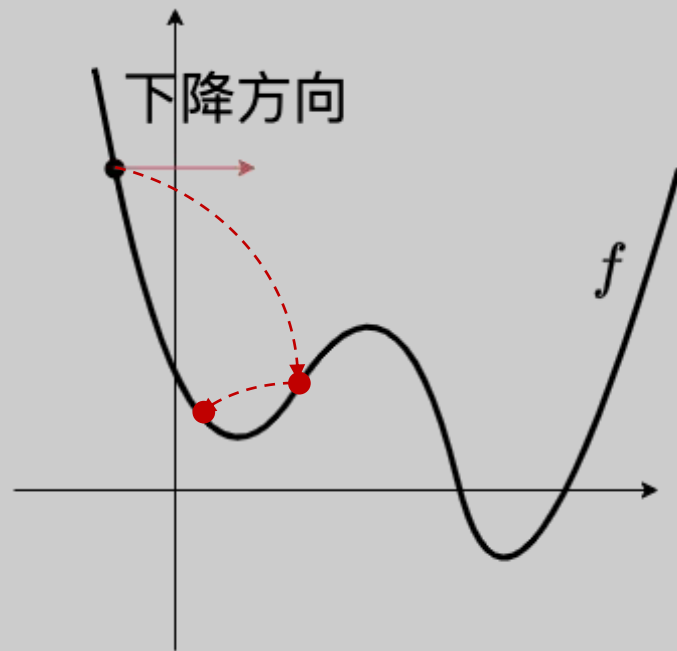


Condition d'optimization :
 $f'(x) = 0$

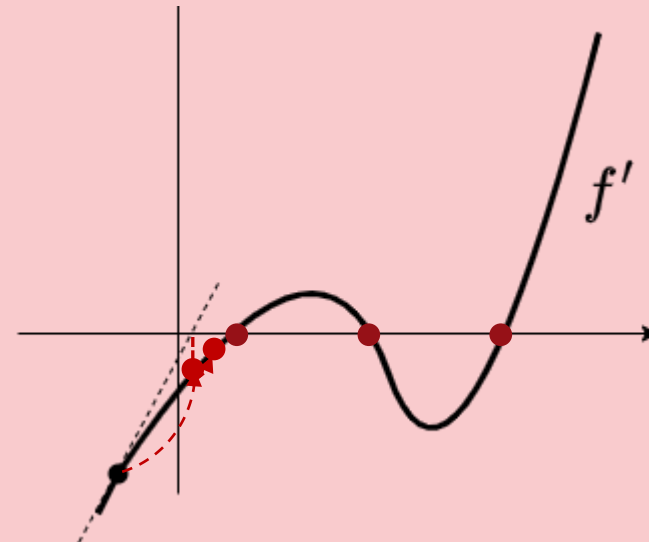


Développement limité

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$



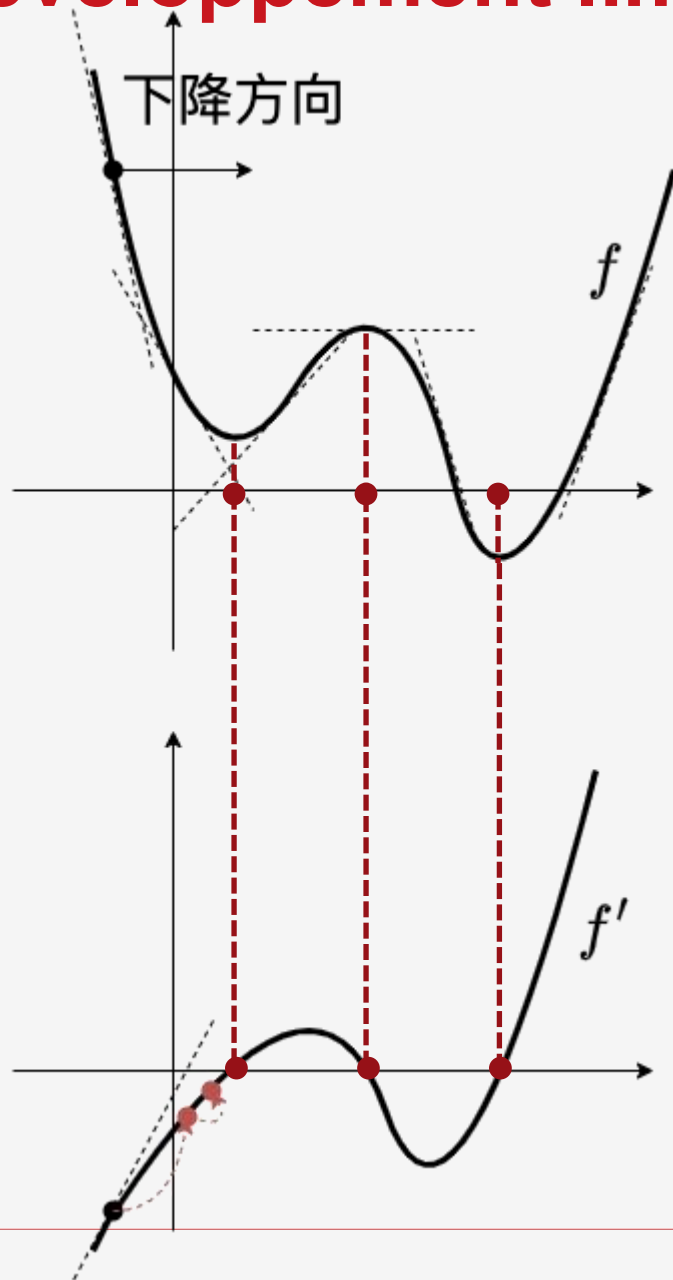
Condition d'optimization :
 $f'(x) = 0$



$$f'(x) \approx f'(x_k) + f''(x_k)(x - x_k) = 0$$

$$x = x_k - \frac{f'(x_k)}{f''(x_k)}$$

Développement limité



Méthode de Newton classique

Pour une fonction $f(x)$ quadratiquement différentiable, considérons l'approximation de Taylor du second ordre de la fonction f objectif au point x_k .

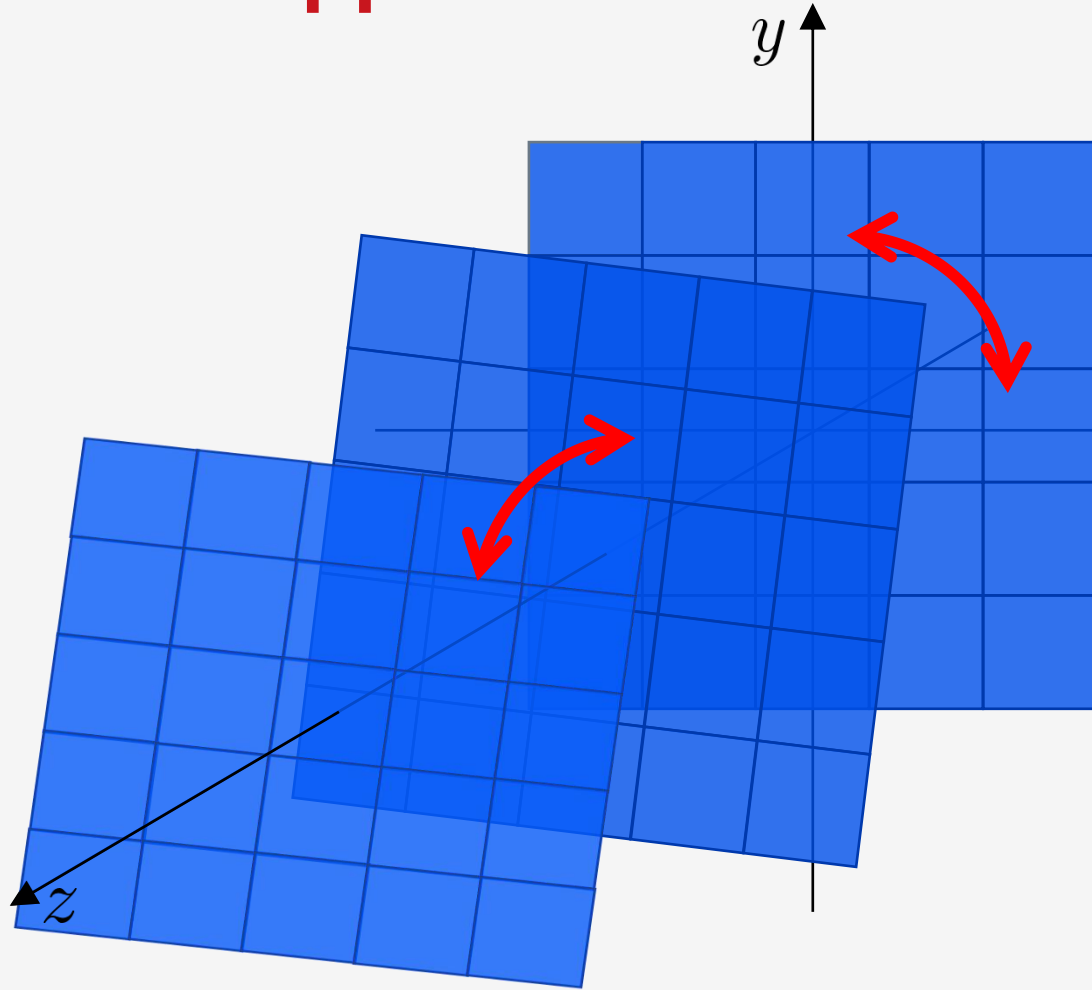
$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2 + o((x - x_k)^2)$$

En négligeant les termes d'ordre supérieur $o((x - x_k)^2)$ et en minimisant la fonction, on obtient

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

Cette méthode itérative est connue sous le nom de **méthode de Newton classique**.

Développement limité



Nous devons utiliser des méthodes de différenciation de dimension supérieure.

Le tremblement comprend une translation et une rotation respectivement le long des axes x , y et z , ayant ainsi six dimensions $(t_x, t_y, t_z, \theta_x, \theta_y, \theta_z)$.

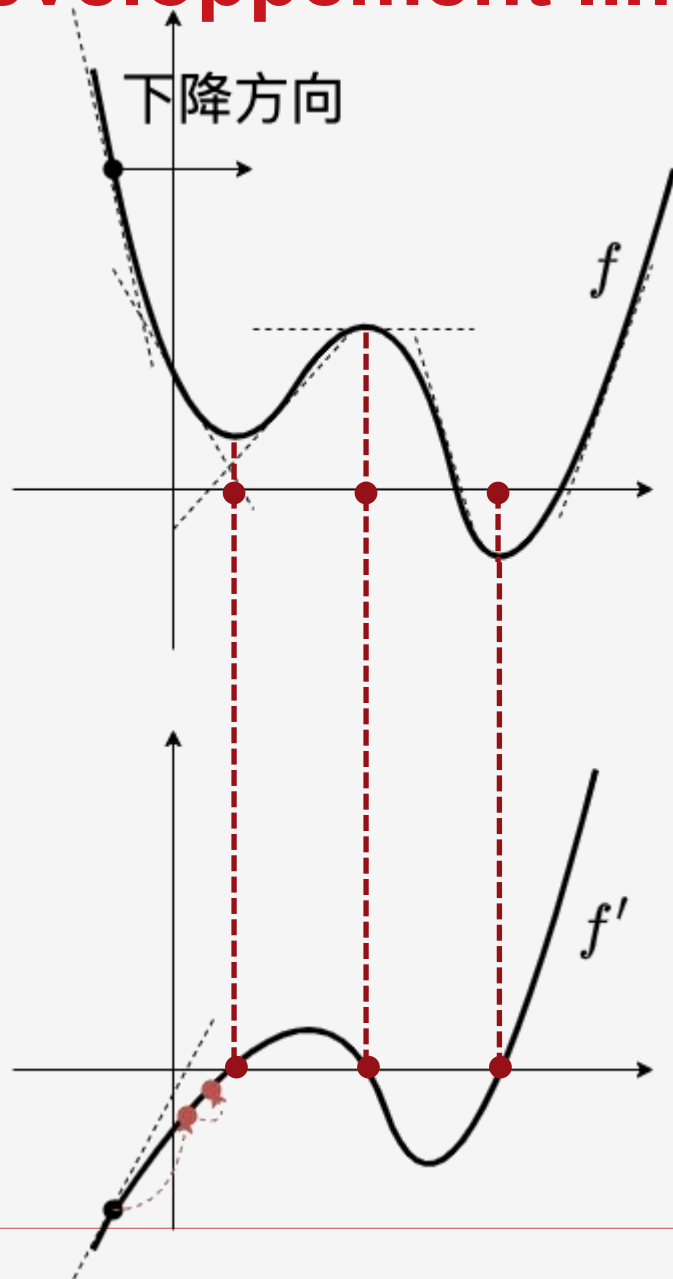
L'**objectif** de l'optimisation est de minimiser la somme des carrés des différences entre chaque pixel de la nouvelle image et l'image de référence

$$f(x) = \sum_{i=1}^N \|I_{\text{ref}}(p_i) - I_{\text{curr}}(p_i + T(x))\|^2$$

où:

- **variable** $x = (t_x, t_y, t_z, \theta_x, \theta_y, \theta_z)$
- N : Nombre total de pixels
- $I_{\text{ref}}(p_i)$: paramètre du i -ième pixel dans l'image ref
- $I_{\text{curr}}(p_i + T(x))$: du i -ième pixel dans l'image per

Développement limité



Méthode de Newton classique

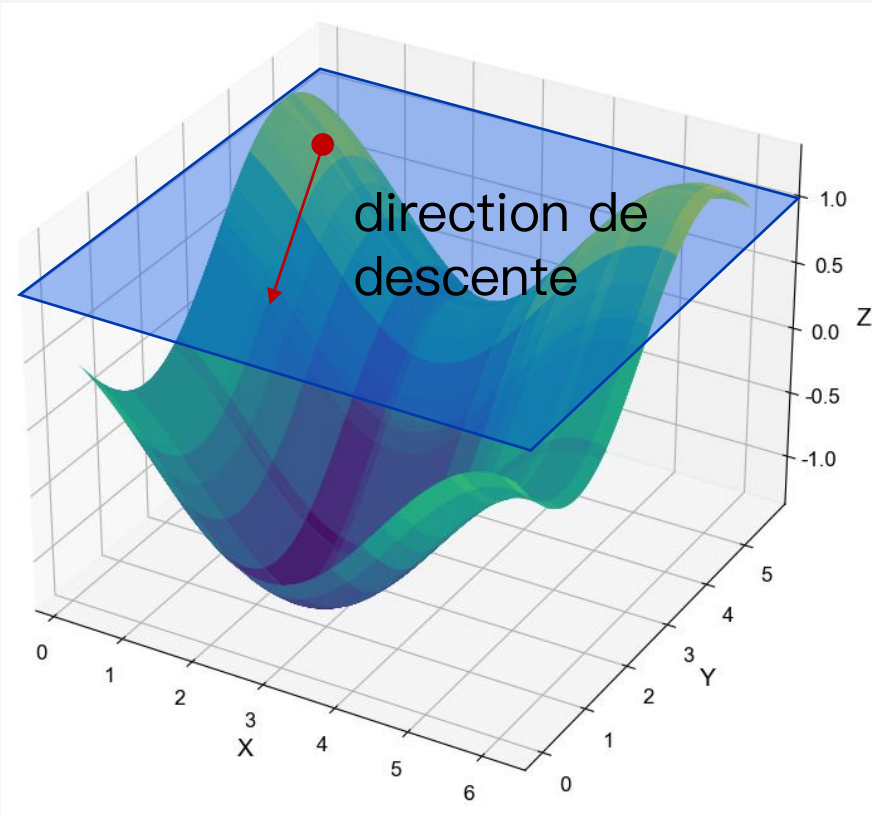
Pour une fonction $f(x)$ quadratiquement différentiable, considérons l'approximation de Taylor du second ordre de la fonction f objectif au point x_k .

$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2 + o((x - x_k)^2)$$

En négligeant les termes d'ordre supérieur $o((x - x_k)^2)$ et en minimisant la fonction, on obtient

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

Cette méthode itérative est connue sous le nom de **méthode de Newton classique**.



Méthode de Newton classique

Pour une fonction $f(x)$ quadratiquement différentiable, considérons l'approximation de Taylor du second ordre de la fonction f objectif au point x_k .

$$f(x) = f(x_k) + \nabla f(x_k)^T (x - x_k) + \frac{1}{2} (x - x_k)^T H_f(x_k) (x - x_k) + o(\|x - x_k\|^2)$$

En négligeant les termes d'ordre supérieur $o((x - x_k)^2)$ et en minimisant la fonction, on obtient

$$x_{k+1} = x_k - H_f(x_k)^{-1} \nabla f(x_k)$$

Cette méthode itérative est connue sous le nom de **méthode de Newton classique**.

Problème d'optimisation sans contrainte $\min_{x \in \mathbb{R}^n} f(x)$

$$\begin{aligned} f(x) &= f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \|x - x_k\| \varepsilon(x - x_k) \\ &= f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2} \langle H_f(x_k) \cdot (x - x_k), x - x_k \rangle + \|x - x_k\|^2 \varepsilon(x - x_k) \end{aligned}$$

Chapitre 4 :
Méthode de (descente de) gradient

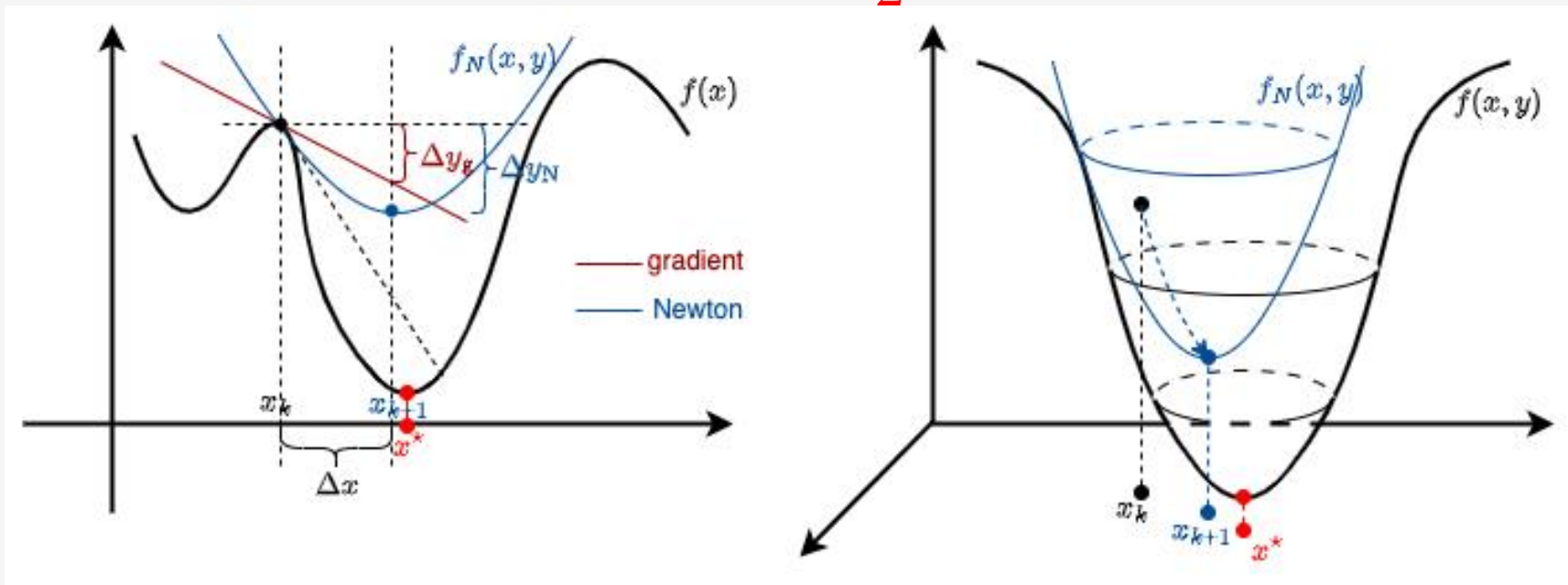
$$x = x_k - \delta \nabla f(x_k)$$

Chapitre 5 :
Méthode de (descente de) Newton

Taylor-Young au second ordre au point x

$$f(x) = f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2} \langle H_f(x_k) \cdot (x - x_k), x - x_k \rangle + \|x - x_k\|^2 \varepsilon(x - x_k)$$

$$\implies g(x) = g(x_k) + \langle \nabla g(x_k), x - x_k \rangle + \frac{1}{2} \langle H_g(x_k) \cdot (x - x_k), x - x_k \rangle$$



$$g(x) = g(x_k) + \langle \nabla g(x_k), x - x_k \rangle + \frac{1}{2} \langle H_g(x_k) \cdot (x - x_k), x - x_k \rangle$$

Cette fonction est donc minimale pour $x = x_k - H_g(x_k)^{-1} \cdot \nabla g(x_k)$



Algorithme de descente de Newton

1

Algorithme de descente général

Algorithme de descente de Newton

Un point initial $x_0 \in \mathbb{R}^n$, un seuil de tolérance $\varepsilon > 0$

Un point $x \in \mathbb{R}^n$ proche de x^*

Initialiser : $x \leftarrow x_0$
 $k \leftarrow 0$

def descente général :

Tant que $\|\nabla f(x)\| \geq \varepsilon$:

1. Trouver une direction de descente d_k
2. Calculer δ_k tel que $\delta_k \nabla f(x_k) \cdot d_k = -\|\nabla f(x_k)\|^2$
3. Mettre à jour x : $x \leftarrow x_{k+1} = x_k + \delta_k \cdot d_k$
 $k \leftarrow k + 1$

Fin tant que

Retourner : x

Condition de résiliation

1. Nombre d'itérations

$$N$$

2. Critère de tolérance

$$|f(x_{k+1}) - f(x_k)|$$

3. Objectif de précision

$$f(x_k)$$

4. Convergence de la fonction de coût

$$f(x_k) = \inf f(x)$$

5. etc.

I. Méthode de LM (Levenberg-Marquardt, 1963)

Problème d'optimisation sans contrainte $\min_{x \in \mathbb{R}^n} f(x)$

$$\begin{aligned} f(x) &= f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \|x - x_k\| \varepsilon(x - x_k) \\ &= f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{1}{2} \langle H_f(x_k) \cdot (x - x_k), x - x_k \rangle + \|x - x_k\|^2 \varepsilon(x - x_k) \end{aligned}$$

Chapitre 4 :
Méthode de (descente de) gradient

$$x = x_k - \delta \nabla f(x_k)$$

Chapitre 5 :
Méthode de (descente de) Newton

$$x = x_k - H_f^{-1}(x_k) \cdot \nabla f(x_k)$$

convergence rapide mais nécessité de calculer la matrice hessienne

Algorithme de descente de Newton

Algorithme de descente de Newton

Un point initial $x_0 \in \mathbb{R}^n$, un seuil de tolérance $\varepsilon > 0$

Un point $x \in \mathbb{R}^n$ proche de x^*

Initialiser : $x \leftarrow x_0$

$k \leftarrow 0$

def descente général :

Tant que $\|\nabla f(x)\| \geq \varepsilon$:

1. Calculer $d_k = -H_f(x_k)^{-1} \cdot \nabla f(x_k)$

3. Mettre à jour x : $x \leftarrow x_{k+1} = x_k + \delta_k \cdot d_k$

$k \leftarrow k + 1$

Fin tant que

Retourner : x

Flou de bougé

Nous prenons actuellement une photo du chiffre « 1 » en blanc sur fond noir.
Pour simplifier les calculs, nous supposons un décalage uniquement dans deux directions planes.

Image originale

0.05	0.00	1.00	0.15	0.00
0.00	0.16	1.00	0.00	0.05
0.00	0.00	1.00	0.00	0.00
0.00	0.00	1.00	0.00	0.00
0.15	0.00	1.00	0.00	0.00

Image tremblante

0.00	0.00	0.16	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.15	0.00	1.00	0.00
0.00	0.00	0.00	0.00	0.00

优化目标为最小化新图像和参考图像每个像素间的差值平方和，因此目标函数为：

$$f(t_x, t_y) = \sum_{i=1}^N \|I_{\text{ref}}(p_i) - I_{\text{curr}}(p_i + T(t_x, t_y))\|^2$$

Flou de bougé

我们现在拍摄一张黑底白字的数字“1”，过程中产生了不可避免的抖动。
为了简化计算，我们假设只有两个平面方向的偏移。

Image originale

0.05	0.00	1.00	0.15	0.00
0.00	0.16	1.00	0.00	0.05
0.00	0.00	1.00	0.00	0.00
0.00	0.00	1.00	0.00	0.00
0.15	0.00	1.00	0.00	0.00

Image tremblante

0.00	0.00	0.16	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.15	0.00	1.00	0.00
0.00	0.00	0.00	0.00	0.00

valeur absolue de
l'erreur d'alignement

0.05	0.00	0.84	0.85	0.00
0.00	0.16	1.00	1.00	0.05
0.00	0.00	1.00	1.00	0.00
0.00	0.15	1.00	1.00	0.00
0.15	0.00	1.00	0.00	0.00

$f(0,0)$

优化目标为最小化新图像和参考图像每个像素间的差值平方和，因此目标函数为：

$$f(0,0) = \sum_{i=1}^N \|I_{\text{ref}}(p_i) - I_{\text{curr}}(p_i + T(0,0))\|^2 = \underbrace{0.05^2 + 0.00^2 + 0.84^2 + \dots + 0.00^2}_{25 \text{ pixels}} = 8.5037$$

Flou de bougé

Image originale					Image tremblante					$f(t_x, t_y)$					
0.05	0.00	1.00	0.15	0.00	0.00	0.00	0.16	1.00	0.00	2	5.10	8.90	8.78	1.13	8.51
0.00	0.16	1.00	0.00	0.05	0.00	0.00	0.00	1.00	0.00	1	5.10	9.62	9.53	0.00	9.51
0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0	5.12	8.68	8.50	1.13	8.75
0.00	0.00	1.00	0.00	0.00	0.00	0.15	0.00	1.00	0.00	-1	5.10	7.70	7.80	2.10	7.76
0.15	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	-2	5.10	6.70	6.75	3.10	6.76

Flou de bougé

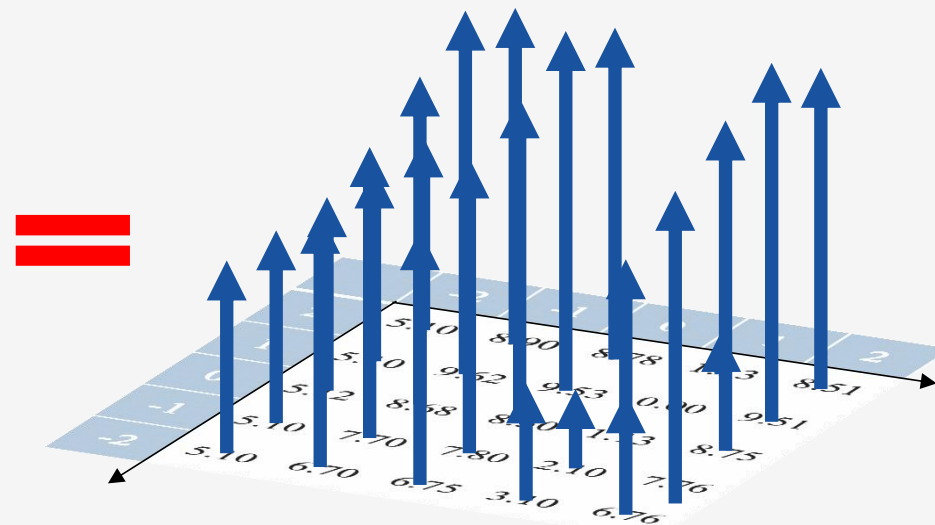
$$f(t_x, t_y)$$

Image originale

0.05	0.00	1.00	0.15	0.00
0.00	0.16	1.00	0.00	0.05
0.00	0.00	1.00	0.00	0.00
0.00	0.00	1.00	0.00	0.00
0.15	0.00	1.00	0.00	0.00

Image tremblante

0.00	0.00	0.16	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.15	0.00	1.00	0.00
0.00	0.00	0.00	0.00	0.00



Flou de bougé

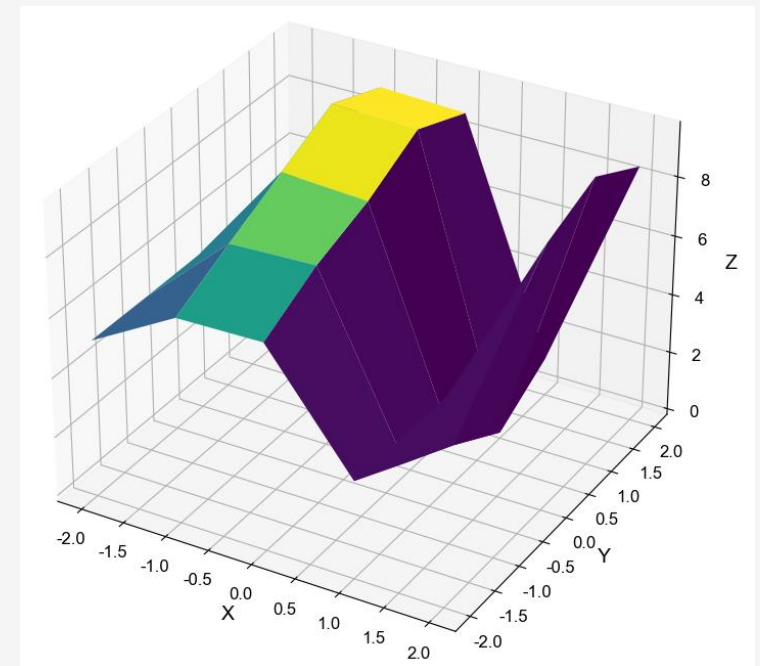
$$f(t_x, t_y)$$

Image originale

0.05	0.00	1.00	0.15	0.00
0.00	0.16	1.00	0.00	0.05
0.00	0.00	1.00	0.00	0.00
0.00	0.00	1.00	0.00	0.00
0.15	0.00	1.00	0.00	0.00

Image tremblante

0.00	0.00	0.16	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.00	0.00	1.00	0.00
0.00	0.15	0.00	1.00	0.00
0.00	0.00	0.00	0.00	0.00



Flou de bougé

$$f(t_x, t_y)$$

	-2	-1	0	1	2
-2	5.10	6.70	6.75	3.10	6.76
-1	5.10	7.70	7.80	2.10	7.76
0	5.12	8.68	8.50	1.13	8.75
1	5.10	9.62	9.53	0.00	9.51
2	5.10	8.90	8.78	1.13	8.51

$$\nabla f(t_x, t_y)$$

	-2	-1	0	1	2
-2	(0,1.6)	(1,0.8)	(1,1.8)	(-1,0)	(1,3.7)
-1	(0,2.6)	(1,1.4)	(1,-3)	(-1,-2)	(1,5.7)
0	(0,3.6)	(1,1.7)	(1,-4)	(-1,1)	(1,7.6)
1	(0,4.5)	(1,2.2)	(0,-5)	(0,-1)	(0,9.5)
2	(0,3.8)	(-1,2)	(-1,-4)	(1,-1)	(-1,7)

$$H_f(t_x, t_y)$$

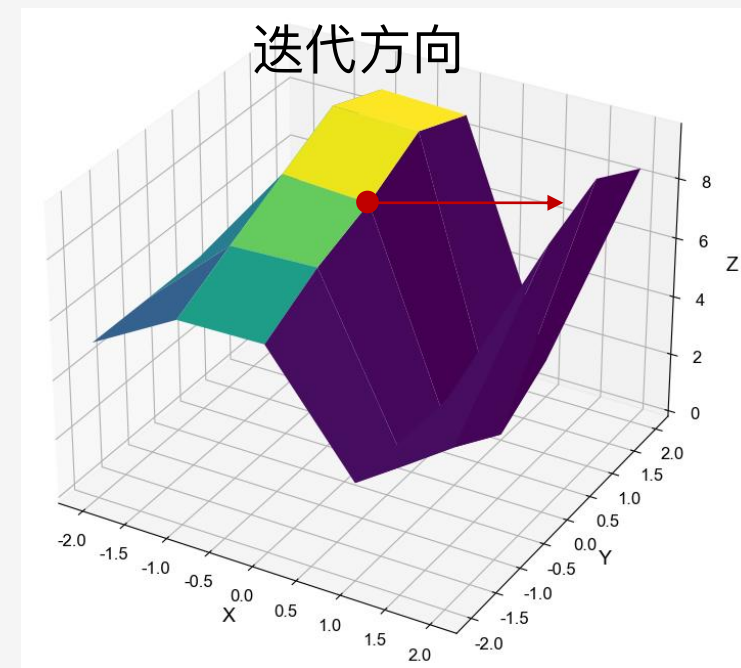
	-2	-1	0	1	2
-2	$\begin{bmatrix} 0.02 & 0.73 \\ 0.73 & -1.96 \end{bmatrix}$	• • •	• • •	• • •	• • •
-1	• • •	• • •	• • •	• • •	• • •
0	• • •	• • •	$\begin{bmatrix} -0.37 & 1.00 \\ 1.00 & -0.78 \end{bmatrix}$	• • •	• • •
1	• • •	• • •	• • •	• • •	• • •
2	• • •	• • •	• • •	• • •	$\begin{bmatrix} -0.01 & -2.00 \\ -2.00 & 3.65 \end{bmatrix}$

Flou de bougé

$$f(t_x, t_y)$$

	-2	-1	0	1	2
-2	5.10	6.70	6.75	3.10	6.76
-1	5.10	7.70	7.80	2.10	7.76
0	5.12	8.68	8.50	1.13	8.75
1	5.10	9.62	9.53	0.00	9.51
2	5.10	8.90	8.78	1.13	8.51

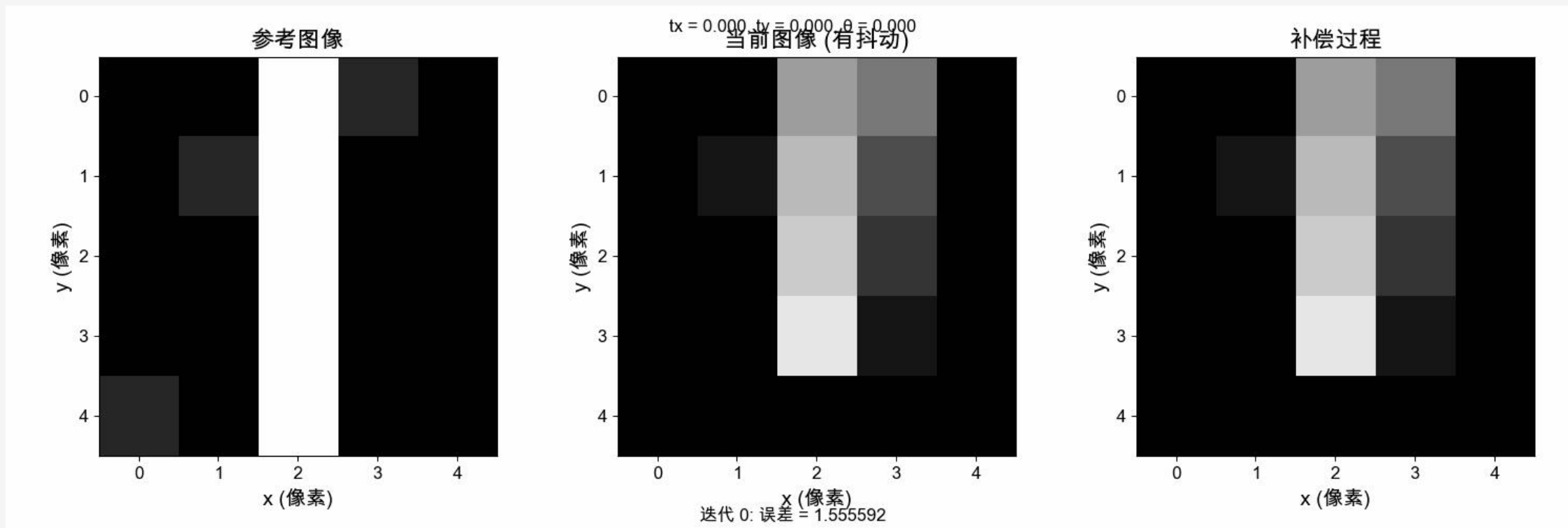
	-2	-1	0	1	2
-2	(0,1.6)	(1,0.8)	(1,1.8)	(-1,0)	(1,3.7)
-1	(0,2.6)	(1,1.4)	(1,-3)	(-1,-2)	(1,5.7)
0	(0,3.6)	(1,1.7)	(1,-4)	(-1,1)	(1,7.6)
1	(0,4.5)	(0,2.2)	(0,-5)	(0,-1)	(0,9.5)
2	(0,3.8)	(-1,2)	(-1,-4)	(1,-1)	(-1,7)



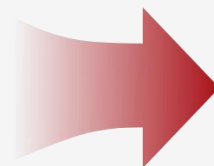
Flou de bougé



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



5 itérations suffisent pour atteindre la convergence



La méthode de Newton est plus efficace pour compenser des vibrations.



Annexe : Méthode de Gauss–Newton

Objectif : Minimiser la somme des carrés des résidus

$$\min_x F(x) = \frac{1}{2} \|r(x)\|^2 = \frac{1}{2} \sum_{i=1}^m r_i(x)^2$$

- $x \in \mathbb{R}^n$: Paramètre à optimiser
- $r(x) \in \mathbb{R}^m$: Vecteur résiduel ($m \geq n$ généralement)
- Cas d'application courants : ajustement de courbes, reconstruction 3D, étalonnage de robots

Méthode de descente de gradient

$$x_{k+1} = x_k - \delta \cdot \nabla F(x_k)$$

- $\nabla F(x) = J(x)^\top r(x)$, où $J_{ij} = \frac{\partial r_i}{\partial x_j}$ (Matrice de Jacobi)

Méthode de descente de Newton

$$x_{k+1} = x_k - H_F(x_k)^{-1} \cdot \nabla F(x_k)$$

- $H_F(x) = J(x)^\top \cdot J(x) + \sum_i r_i \cdot H_{r_i}(x)$

Méthode de Gauss-Newton

$$H_F(x) \approx J(x)^\top \cdot J(x)$$

$$(J(x)^\top \cdot J(x)) \Delta = -J(x)^\top \cdot r(x)$$

$$x_{k+1} = x_k + \Delta$$

Méthode de LM (Levenberg-Marquardt, 1963)

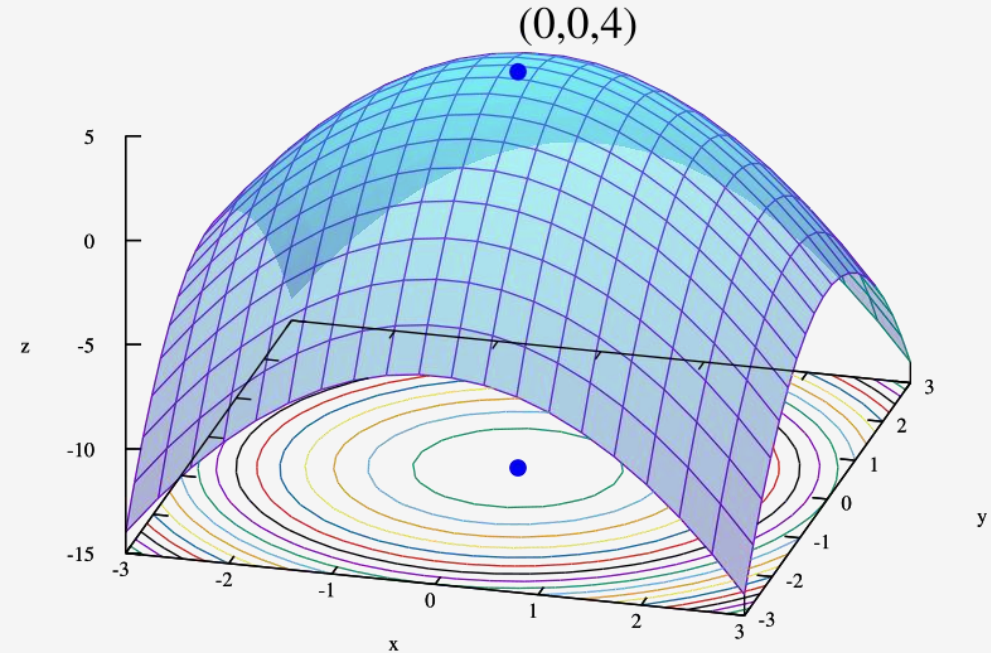
- Combinaison des avantages de la descente de gradient et de la méthode de Newton
- Résolution du problème de singularité de la hessienne
- Adaptée aux problèmes de moindres carrés non linéaires

$$\text{LM: } (J(x)^\top \cdot J(x) + \lambda \cdot I) \cdot \Delta = -J(x)^\top \cdot r(x)$$

$$\lambda = 0$$



Gauss-Newton

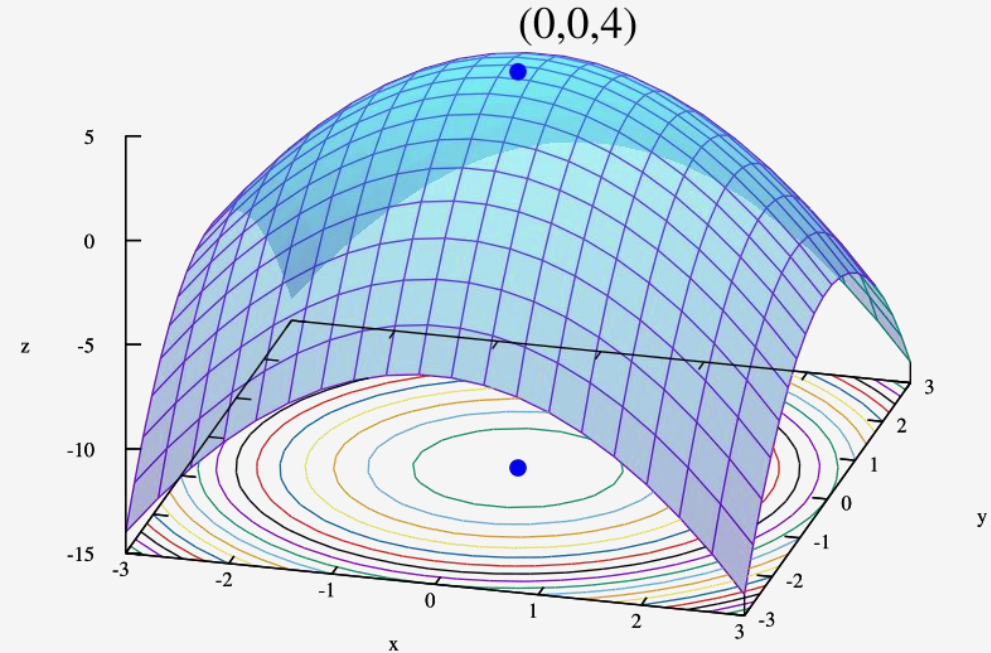


https://en.wikipedia.org/wiki/File:Max_paraboloid.svg

Méthode de LM (Levenberg-Marquardt, 1963)

- Combinaison des avantages de la descente de gradient et de la méthode de Newton
- Résolution du problème de singularité de la hessienne
- Adaptée aux problèmes de moindres carrés non linéaires

$$\text{LM: } (J(x)^\top \cdot J(x) + \lambda \cdot I) \cdot \Delta = -J(x)^\top \cdot r(x)$$



https://en.wikipedia.org/wiki/File:Max_paraboloid.svg



Algorithme de quasi-Newton

2

Flou de bougé



Optimisation



impose des exigences élevées
méthode de Newton



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Google Photos : Post-traitement de la stabilisation vidéo
Module complémentaire de stabilisation d'image pour les logiciels
de montage vidéo Adobe

trouver un équilibre entre vitesse de convergence
et coût de calcul : **méthodes quasi-Newton**

Problème de la méthode de Newton

Méthode de Newton

01 Gradient

02 Matrice hessienne

03 Renouveler

$$x_{k+1} = x_k - H_f(x_k)^{-1} \nabla f(x_k)$$

Par exemple

$$f(x, y) = x^4 + y^2$$

Calculer la matrice hessienne au point $(0, 0)$

$$H_f(0, 0) = \begin{bmatrix} 12x^2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$$

Calculer l'inverse par la méthode de pivot de Gauss

$$\Rightarrow \left[\begin{array}{cc|cc} 0 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \end{array} \right] \Rightarrow \left[\begin{array}{cc|cc} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & \frac{1}{2} \end{array} \right]$$

$H_f(0, 0)$ **l'inverse n'existe pas !**

La méthode de Newton nécessite le calcul de la matrice hessienne.

Dans les cas de stabilisation vidéo, si une optimisation au niveau des pixels (mégapixels) est effectuée, les coûts de calcul et de stockage de la matrice hessienne sont élevés, et il peut être impossible de calculer sa matrice inverse.

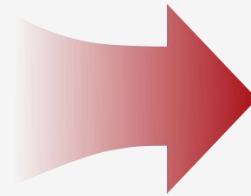


Problème de la méthode de Newton

Méthode de Newton

- 01 Gradient
- 02 Matrice hessienne
- 03 Renouveler

$$x_{k+1} = x_k - H_f(x_k)^{-1} \nabla f(x_k)$$



Améliorations apportées à la méthode de Newton

- 01 Gradient
- 02 Matrice hessienne approx ✓ **inversible**
✓ **facile**
- 03 Renouveler

$$x_{k+1} = x_k - H_k^{-1} \nabla f(x_k)$$

La méthode de Newton nécessite le calcul de la matrice hessienne.

Dans les cas de stabilisation vidéo, si une optimisation au niveau des pixels (mégapixels) est effectuée, les coûts de calcul et de stockage de la matrice hessienne sont élevés, et il peut être impossible de calculer sa matrice inverse.



Formule générale des méthodes de quasi-Newton

convergence rapide mais nécessite de calculer la matrice hessienne

$$x_{k+1} = x_k - H_f^{-1}(x_k) \cdot \nabla f(x_k)$$

||| ➡ Solution : approximation de l'inverse de la hessienne (Principe des méthodes de quasi-Newton)

$$x_{k+1} - H_f^{-1}(x_k) \cdot \nabla f(x_k) = x_k - H_k \cdot \nabla f(x_k)$$

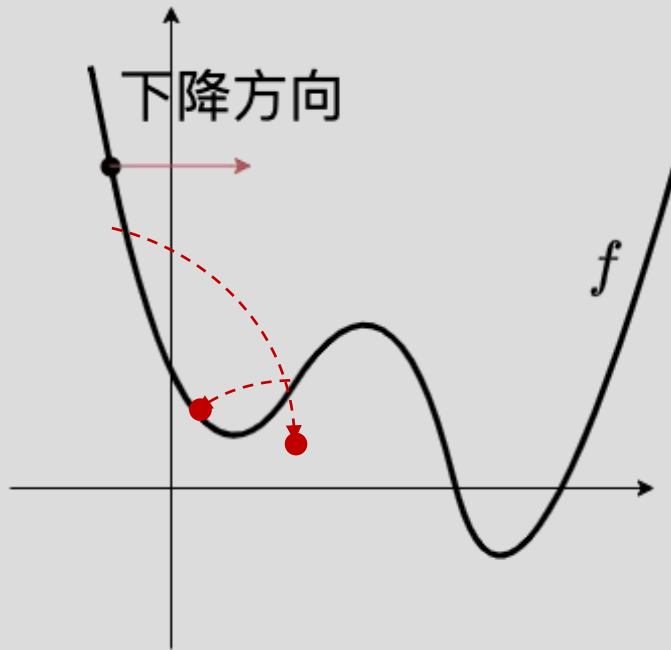
Question : Comment trouver l'approximation de l'inverse de la hessienne?

Formule générale des méthodes de quasi-Newton

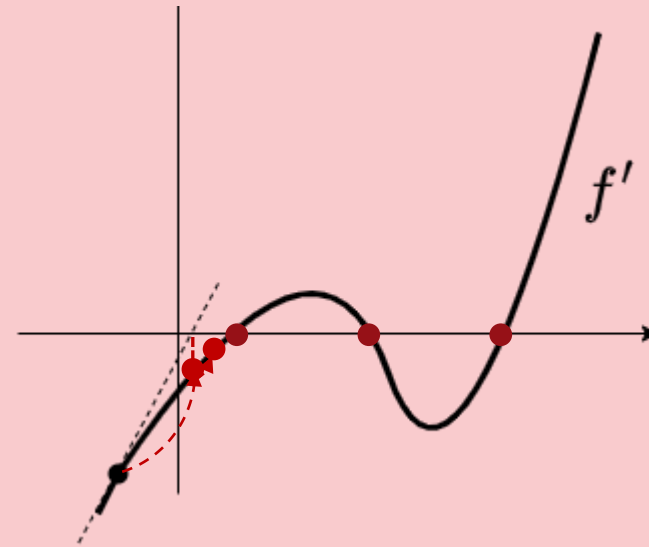


上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$



Condition d'optimization :
 $f'(x) = 0$

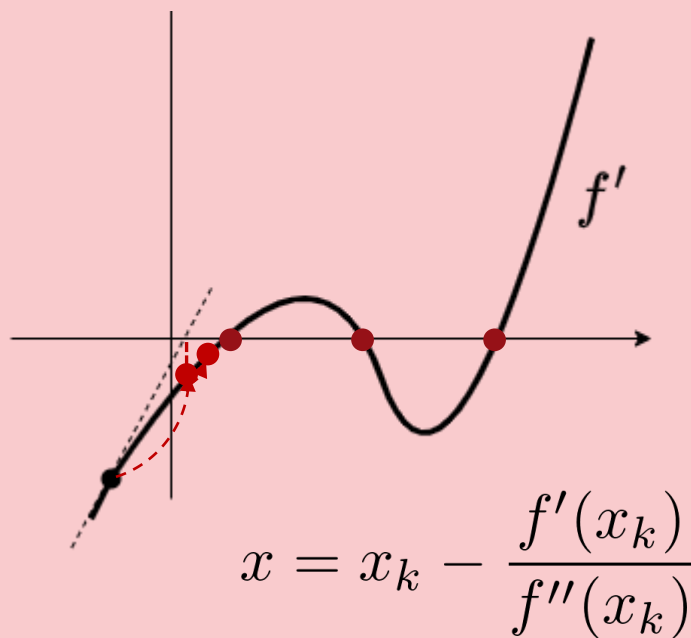


$$f'(x) \approx f'(x_k) + f''(x_k)x - x_k f''(x_k) = 0$$

$$x = x_k - \frac{f'(x_k)}{f''(x_k)}$$

Formule générale des méthodes de quasi-Newton

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$



Pour simplifier le calcul, il faut éviter d'utiliser la dérivée seconde !

Approximer la dérivée seconde à l'aide de la méthode des différences finies.

$$f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}} = b_k$$

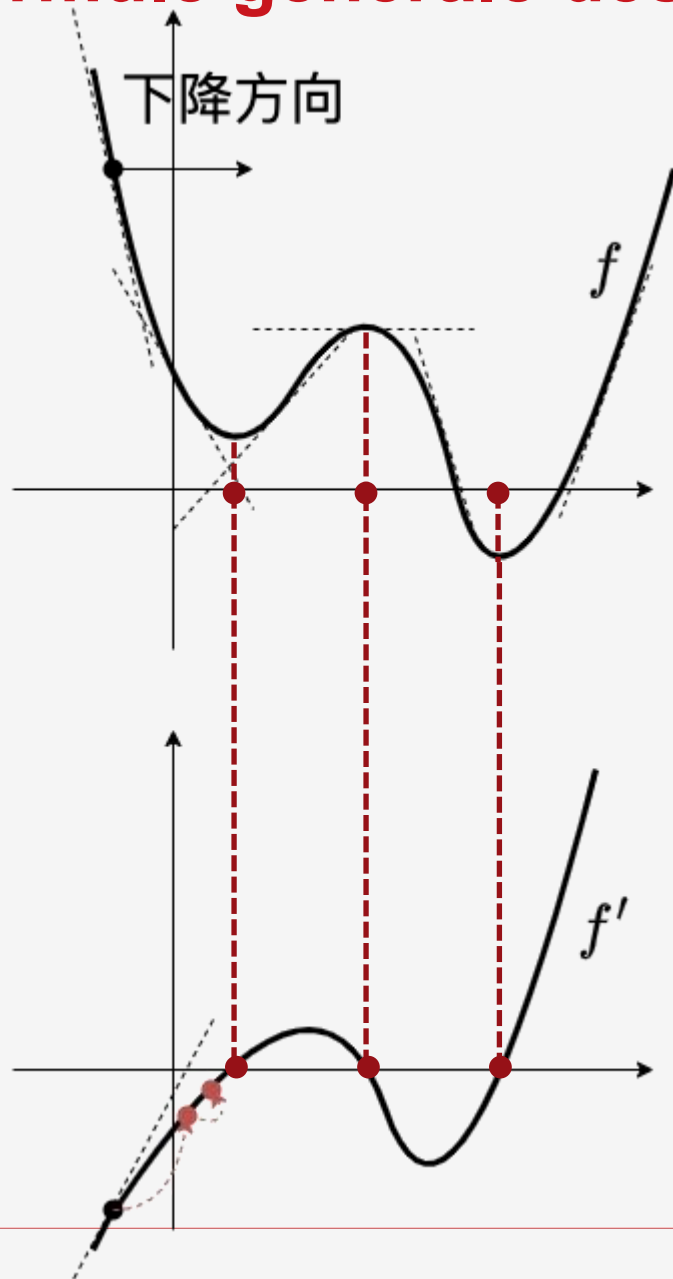
Substituer dans la formule itérative

$$\begin{aligned} x &= x_k - \frac{f'(x_k)}{b_k} = x_k - \frac{f'(x_k)}{\frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}} \\ &= x_k - \frac{f'(x_k)(x_k - x_{k-1})}{f'(x_k) - f'(x_{k-1})} \end{aligned}$$

Formule générale des méthodes de quasi-Newton



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Méthode de quasi-Newton

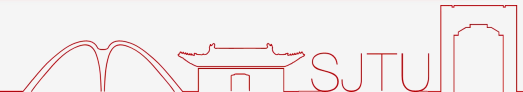
Pour une fonction $f(x)$ quadratiquement différentiable, considérons l'approximation de Taylor du second ordre de la fonction f objectif au point x_k .

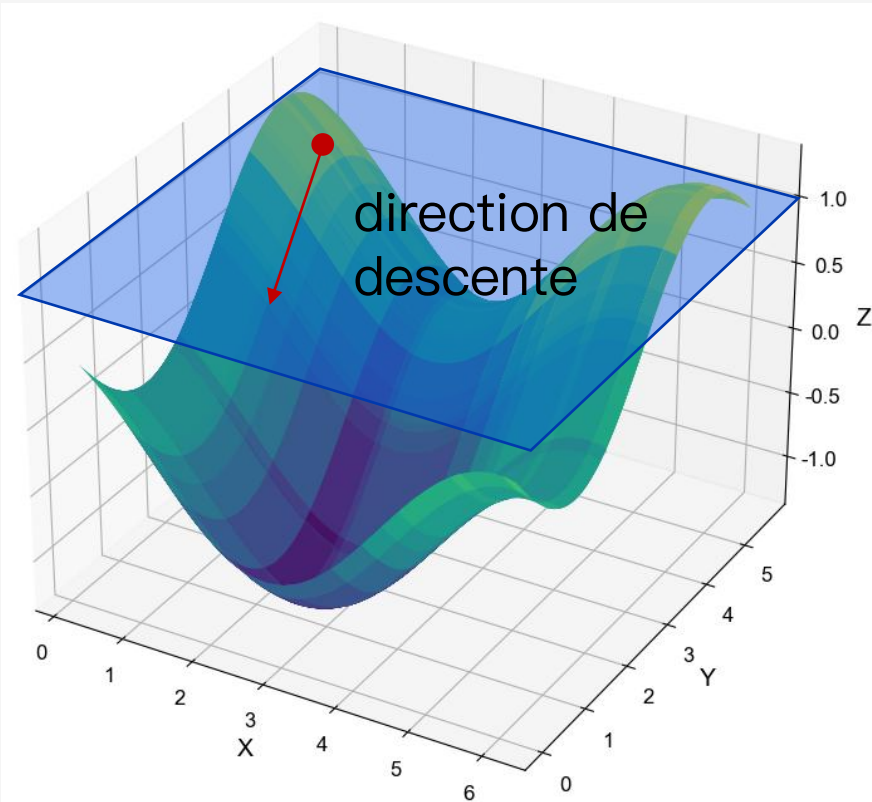
$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}b_k(x - x_k)^2 + o((x - x_k)^2)$$

En négligeant les termes d'ordre supérieur $o((x - x_k)^2)$ et en minimisant la fonction, on obtient

$$x_{k+1} = x_k - \frac{f'(x_k)}{b_k} = x_k - \frac{f'(x_k)(x_k - x_{k-1})}{f'(x_k) - f'(x_{k-1})}$$

Cette méthode itérative est connue sous le nom de **méthode de quasi-Newton**.





Méthode de Newton

Pour une fonction $f(x)$ quadratiquement différentiable, considérons l'approximation de Taylor du second ordre de la fonction f objectif au point x_k .

$$f(x) = f(x_k) + \nabla f(x_k)^T (x - x_k) + \frac{1}{2} (x - x_k)^T \boxed{H_f(x_k)} (x - x_k) + o(\|x - x_k\|^2)$$

En négligeant les termes d'ordre supérieur $o((x - x_k)^2)$ et en minimisant la fonction, on obtient

$$x_{k+1} = x_k - \boxed{H_f(x_k)^{-1}} \nabla f(x_k)$$

Cette méthode itérative est connue sous le nom de **méthode de Newton classique**.

Formule générale des méthodes de quasi-Newton

$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$$

Comment trouver B_k ?

$$\nabla f(x_k) = \nabla f(x_{k+1}) + H_f(x_{k+1}) \cdot (x_k - x_{k+1}) + \mathcal{O}(\|x_k - x_{k+1}\|^2)$$

$$\nabla f(x_{k+1}) - \nabla f(x_k) = H_f(x_{k+1}) \cdot (x_{k+1} - x_k) + \mathcal{O}(\|x_k - x_{k+1}\|^2)$$

Formule générale des méthodes de quasi-Newton



$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$$

Comment trouver B_k ?

$$\underbrace{\nabla f(x_{k+1}) - \nabla f(x_k)}_{\gamma_k} = H_f(x_{k+1}) \cdot \underbrace{(x_{k+1} - x_k)}_{\Delta_k} + \mathcal{O}(\|x_k - x_{k+1}\|^2)$$



Formule générale des méthodes de quasi-Newton

$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$$

Comment trouver B_k ?

$$\nabla f(x_{k+1}) - \nabla f(x_k) = H_f(x_{k+1}) \cdot (x_{k+1} - x_k) + \mathcal{O}(\|x_k - x_{k+1}\|^2)$$

$$\gamma_k = H_f(x_{k+1}) \cdot \Delta_k + \mathcal{O}(\|\Delta_k\|^2)$$

Formule générale des méthodes de quasi-Newton

$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$$

Comment trouver B_k ?

$$\gamma_k = B_{k+1} \cdot \Delta_k$$

Assurer l'inversibilité

$$B_{k+1} = B_k + a_k \cdot u_k \cdot u_k^T$$

$$\gamma_k = (B_k + a_k \cdot u_k \cdot u_k^T) \cdot \Delta_k \Rightarrow \gamma_k - B_k \cdot \Delta_k = a_k \cdot u_k \cdot u_k^T \cdot \Delta_k$$

Formule générale des méthodes de quasi-Newton



$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$$

Comment trouver B_k ?

$$\gamma_k - B_k \cdot \Delta_k = a_k \cdot u_k \cdot u_k^T \cdot \Delta_k$$

1. Si $\gamma_k - B_k \cdot \Delta_k = 0$, alors $B_{k+1} = B_k$

2. Si $\gamma_k - B_k \cdot \Delta_k \neq 0$, alors $a_k = \frac{1}{u_k^T \cdot \Delta_k}$ et $u_k = \gamma_k - B_k \cdot \Delta_k$

$$B_{k+1} = B_k + \frac{(\gamma_k - B_k \cdot \Delta_k) \cdot (\gamma_k - B_k \cdot \Delta_k)^T}{(\gamma_k - B_k \cdot \Delta_k)^T \cdot \Delta_k}$$

Formule générale des méthodes de quasi-Newton

Question : Comment trouver l'approximation de l'inverse de la hessienne?

L'approximation de Taylor d'ordre 1

$$\nabla f(x) = \nabla f(x_{k+1}) + H_f(x_{k+1}) \cdot (x - x_{k+1}) + \mathcal{O}(\|x - x_{k+1}\|^2)$$

Si $x = x_k$, $\Delta_k = x_{k+1} - x_k$, $\gamma_k = \nabla f(x_{k+1}) - \nabla f(x_k)$

$$H_f(x_{k+1}) \cdot \Delta_k + \mathcal{O}(\|\Delta_k\|^2) = \gamma_k$$

1. Lorsque $H_f^{-1}(x_k)$ est difficile à résoudre, on utilise l'algorithme DFP (Davidson-Fletcher-Powell)

$$G_k \approx H_k^{-1}$$
$$G_{k+1} \cdot \gamma_k = \Delta_k$$

2. Lorsque $H_f(x_k)$ ou $H_f^{-1}(x_k)$ n'existe pas, on utilise l'algorithme BFGS (Broyden-Fletcher-Goldfarb-Shanno)

$$B_k \approx H_k$$
$$B_{k+1} \cdot \Delta_k = \gamma_k$$

Méthode de Quasi-Newton : DFP

Algorithme DFP (Davidson-Fletcher-Powell, 1964)

L'approximation de l'inverse de la hessienne $G_{k+1} \approx H_{k+1}^{-1} = H_f^{-1}(x_{k+1})$

$$G_{k+1} = G_k + \frac{\Delta_k \cdot \Delta_k^\top}{\Delta_k^\top \cdot \gamma_k} - \frac{G_k \cdot \gamma_k \cdot \gamma_k^\top \cdot G_k}{\gamma_k^\top \cdot G_k \cdot \gamma_k}$$

Méthode de Quasi-Newton : BFGS

Algorithme BFGS (**B**royden-**F**letcher-**G**oldfarb-**S**hanno, 1970)



L'algorithme BFGS tire son nom de quatre mathématiciens spécialistes de l'optimisation.

L'approximation de l'inverse de la hessienne $B_{k+1} \approx H_{k+1} = H_f(x_{k+1})$

$$B_{k+1} = B_k + \frac{\gamma_k \cdot \gamma_k^\top}{\gamma_k^\top \cdot \Delta_k} - \frac{B_k \cdot \Delta_k \cdot \Delta_k^\top \cdot B_k}{\Delta_k^\top \cdot B_k \cdot \Delta_k}$$

Méthode de Quasi-Newton : L-BFGS

Algorithme L-BFGS (Limited-memory BFGS, 1980)

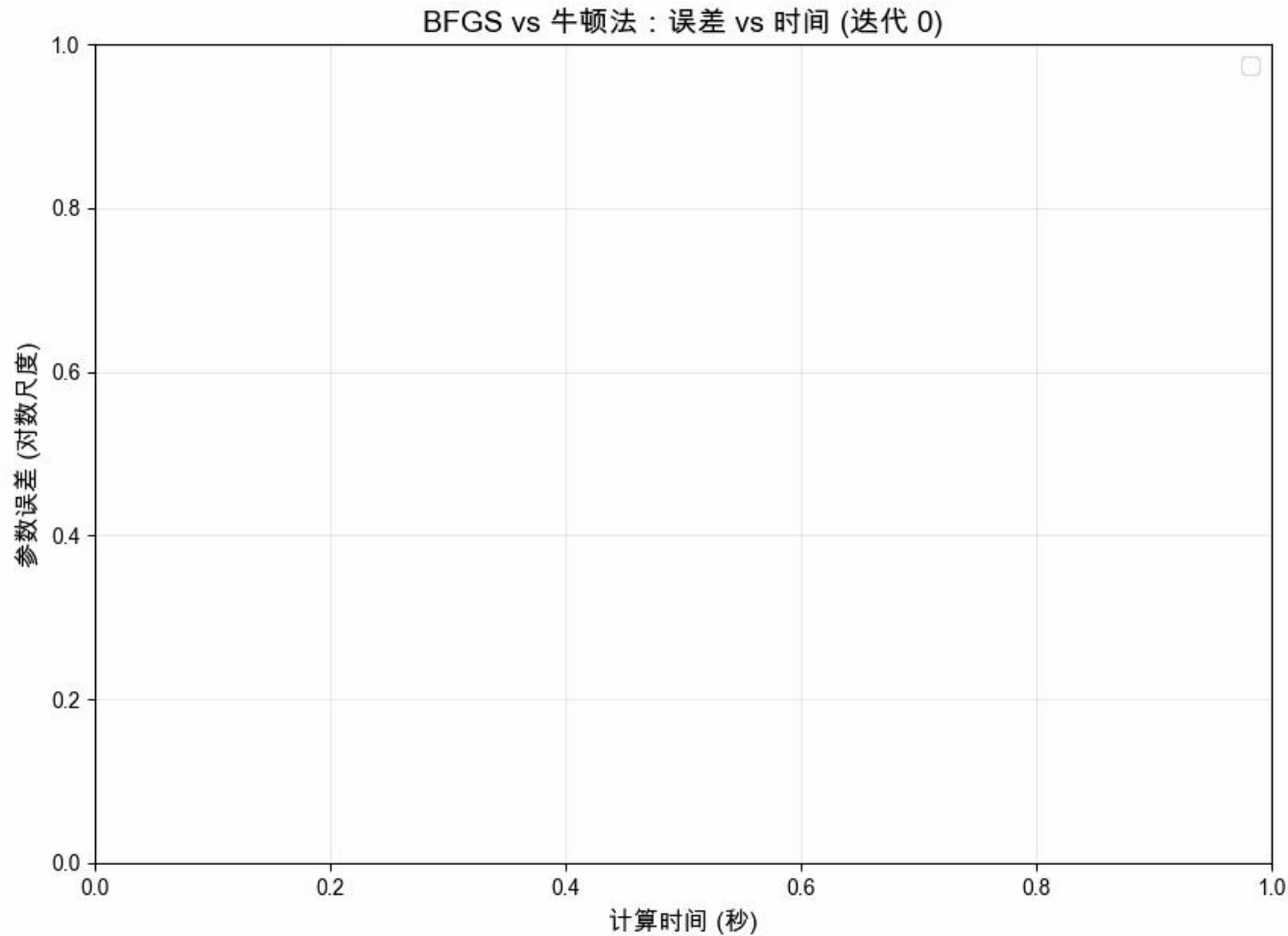
Basé sur l'algorithme BFGS

$$H_{k+1}^{\text{BFGS}} = H_k + \left(1 + \frac{\gamma^\top \cdot H_k \cdot \gamma_k}{\Delta_k^\top \cdot \gamma_k}\right) \frac{\Delta_k \cdot \Delta_k^\top}{\Delta_k^\top \cdot \gamma_k} - \frac{\Delta_k \cdot \gamma_k^\top \cdot H_k + H_k \cdot \gamma_k \cdot \Delta_k^\top}{\Delta_k^\top \cdot \gamma_k}$$

Par itération,

$$\begin{aligned} H_{k+1} &= V_k^\top \cdot V_{k-1}^\top \cdots V_{k-m+1}^\top \cdot H_0 \cdot V_{k-m+1} \cdots V_{k-1} \cdot V_k \\ &+ V_k^\top \cdots V_{k-m+2}^\top \cdot \rho_{k-m+1} \cdot \Delta_{k-m+1} \cdot \Delta_{k-m+1}^\top \cdot V_{k-m+2} \cdots V_k \\ &+ \cdots \\ &+ V_k^\top \cdot \rho_{k-1} \cdot \Delta_{k-1} \cdot \Delta_{k-1}^\top \cdot V_k \\ &+ \rho_k \cdot \Delta_k \cdot \Delta_k^\top \end{aligned}$$

Newton vs quasi-Newton



BFGS:

- 8 itérations
- erreur final = 0.440387
- temps total = 0.0004 s

Newton:

- 8 itérations
- erreur final = 0.426271
- temps total = 0.8330 s



Algorithme de Momentum

3

Problèmes liés aux LLMs



ChatGPT

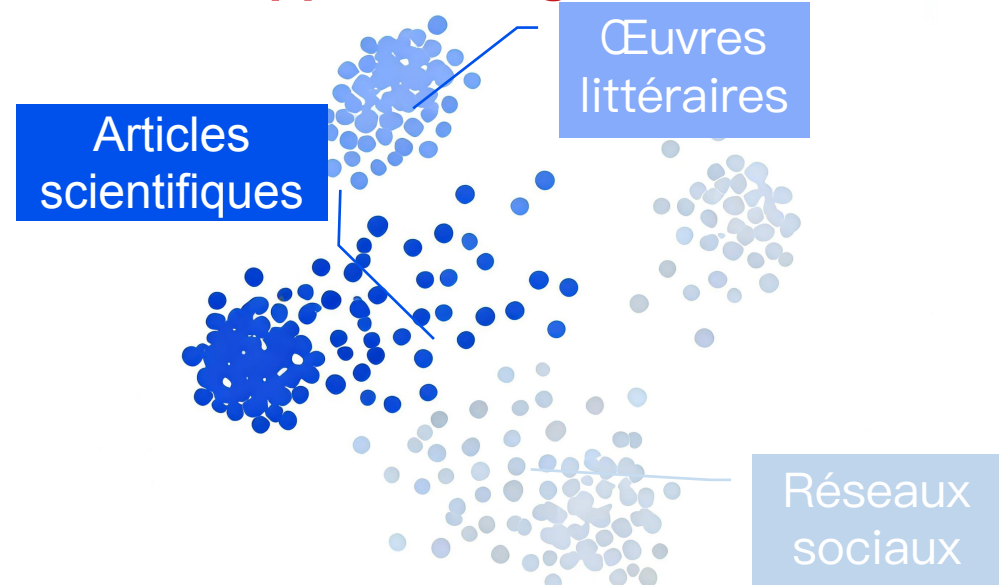
LLaMA
by Meta

Apprentissage distribuée traditionnelle



Données indépendantes et
identiquement distribuées

Apprentissage fédéral



Les données ne sont pas
indépendantes

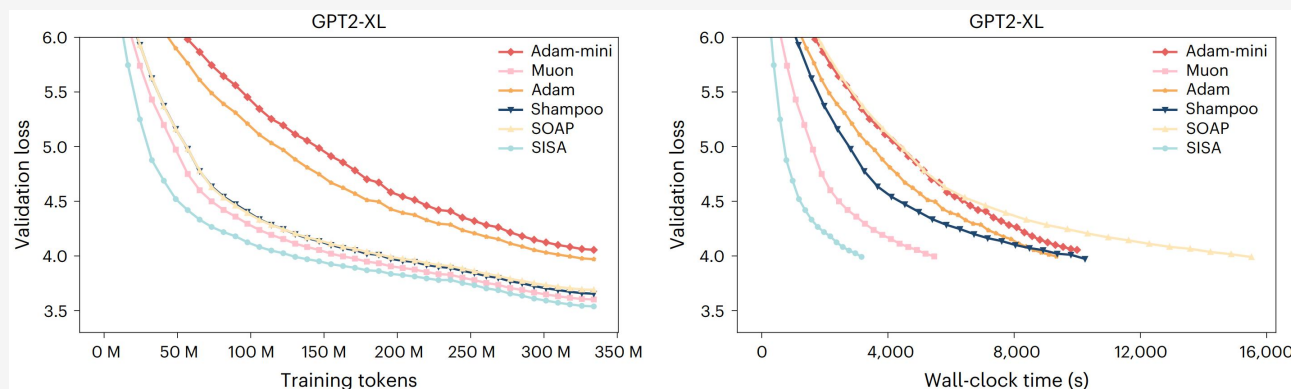
hétérogénéité des données

vitesse de
convergence lente

Faible capacité de
généralisation

Stabilité
d'entraînement réduite

Preconditioned inexact stochastic ADMM for deep models



La vitesse de convergence est nettement améliorée par rapport aux autres algorithmes.

Innovation fondamentale :
Utilisation des mécanismes de dynamique

Zhou S, Wang O, Luo Z, et al. Preconditioned inexact stochastic ADMM for deep models[J]. Nature Machine Intelligence, 2026: 1-12.

nature machine intelligence

Article

<https://doi.org/10.1038/s42256-026-01182-3>

Preconditioned inexact stochastic ADMM for deep models

Ouya Wang^{1,2}, Ziyao Luo¹, Yongxu Zhu² &

nature
machine
intelligence

els are usually trained with stochastic gradient
rithms, but these optimizers face inherent limitations,
rgence and stringent assumptions for convergence.
eterogeneity arising from distributed settings poses
ges to their theoretical and numerical performance.
algorithm called PISA (preconditioned inexact
ing direction method of multipliers). Grounded in
il guarantees, the algorithm converges under the sole
chitz continuity of the gradient on a bounded region,
he need for other conditions commonly imposed
ods. This capability enables the proposed algorithm
nge of data heterogeneity effectively. Moreover,
hitecture enables scalable parallel computing and
reconditions, such as second-order information,
d orthogonalized momentum by Newton–Schulz
rating the last two preconditions in PISA yields two
ficient variants: SISA and NSISA. Comprehensive
iations for training or fine-tuning diverse deep models,
odels, large language models, reinforcement learning
adversarial networks and recurrent neural networks,
ior numerical performance of SISA and NSISA
ious state-of-the-art optimizers.

(SGD)-based optimizers due to their inherent limitations. In this sec-
tion, we review a subset of these methods, sufficient to motivate the
development of our proposed approach.

SGD-based learning algorithms
It is known that the plain SGD-based algorithms, such as the original sto-
chastic approximation approaches^{1,2}, the parallelized SGD³ for machine
learning and the newly developed federated averaging (FedAvg⁴) and
local SGD (LocalSGD⁵) algorithms for federated learning, are frequently
prone to high sensitivity to poor conditioning^{6,7} and slow convergence

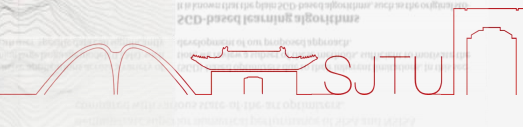
¹TP Laboratory, Department of EEE, Imperial College London,
at University, Nanjing, China. ²Se-mail: shizhou@sjtu.edu.cn

234

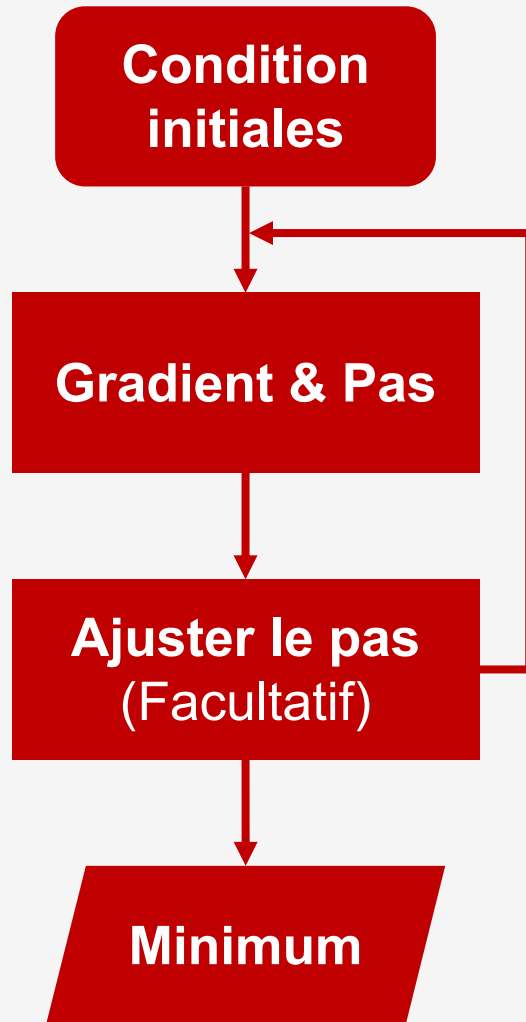
334

A foundation
model for cardiac
health monitoring

health monitoring
model for cardiac



Rappel : descente de gradient



Algorithme général de descente

Un point initial $x_0 \in \mathbb{R}^n$, un seuil de tolérance $\varepsilon > 0$

Un point $x \in \mathbb{R}^n$ proche de x^*

Initialiser : $x \leftarrow x_0$

$k \leftarrow 0$

def descente général :

Tant que $\|\nabla f(x)\| \geq \varepsilon$:

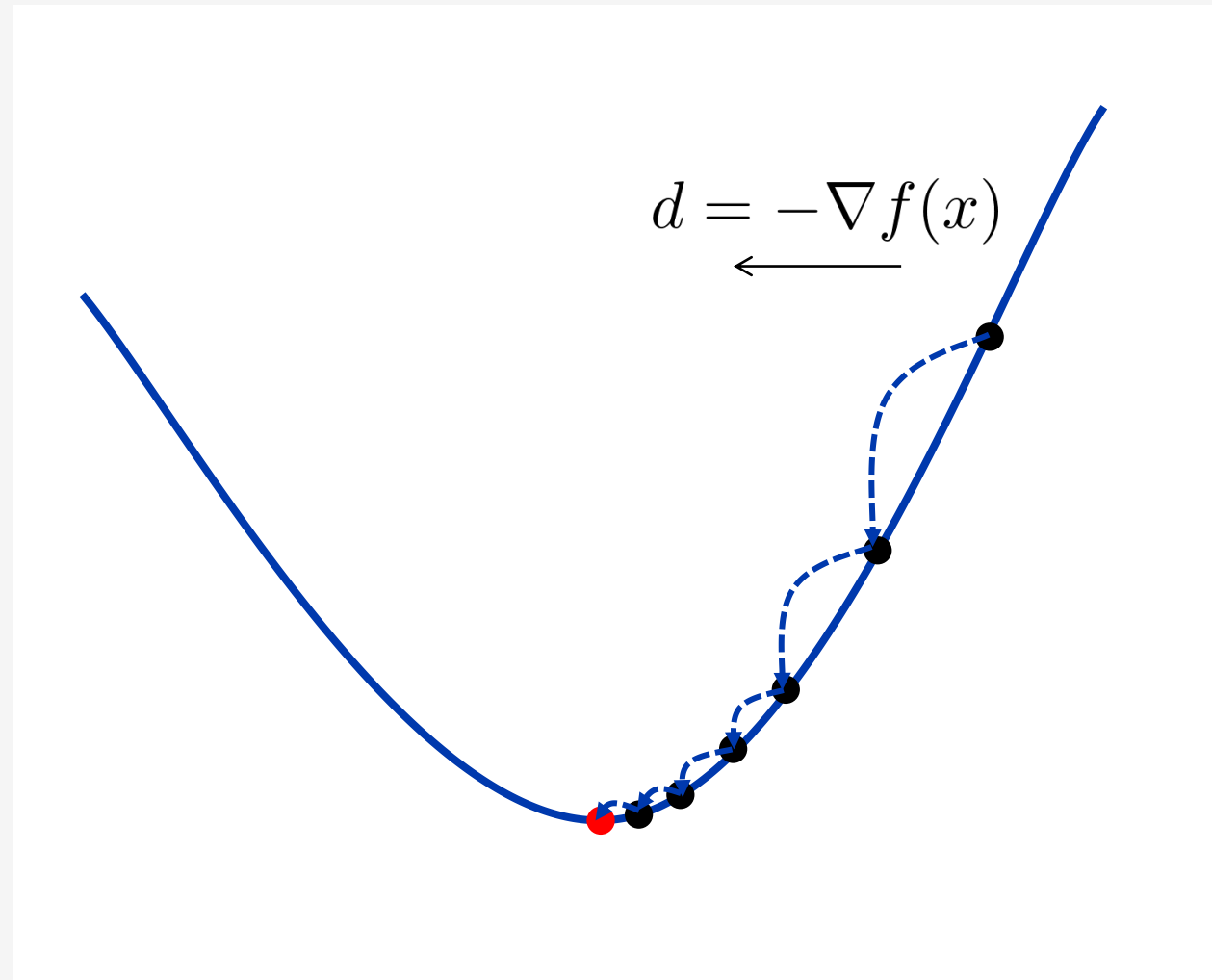
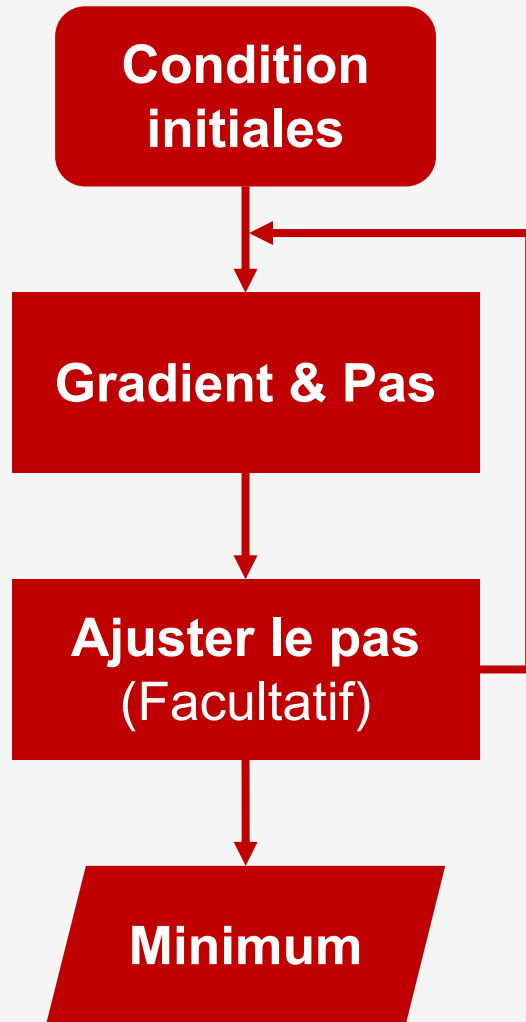
1. Déterminer une direction de descente $d_k \in \mathbb{R}^n$
2. Déterminer un pas de descente $\delta_k > 0$ tel que $f(x_k + \delta_k \cdot d_k) < f(x_k)$
3. Mettre à jour x : $x \leftarrow x_{k+1} = x_k + \delta_k \cdot d_k$

$k \leftarrow k + 1$

Fin tant que

Retourner : x

Le problème de « l'oscillation du canyon »



L'idée principale est de faire un petit pas dans la direction opposée à la gradient à chaque fois.

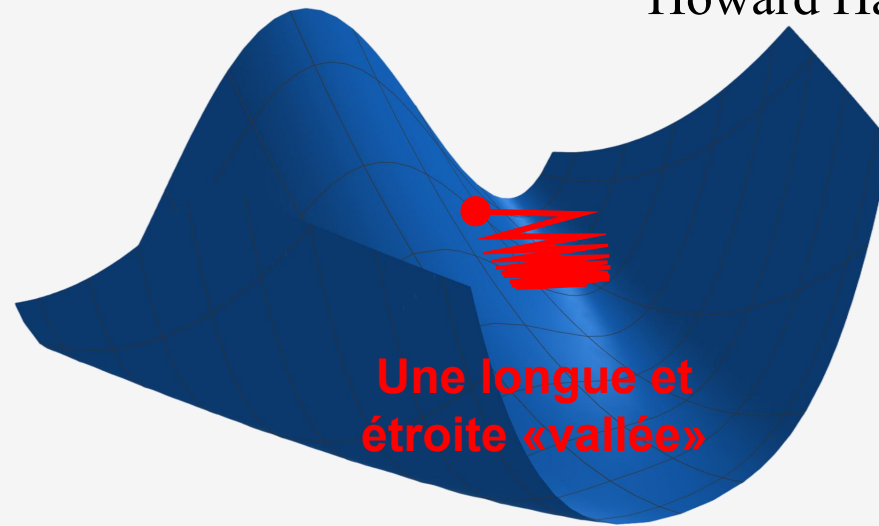
Le problème de « l'oscillation du canyon »



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

Proposer une fonction non convexe pour tester les performances des algorithmes d'optimisation : $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$

— Howard Harry Rosenbrock, 1960



- **Problème d'oscillation** : le gradient oscille rapidement dans les deux sens.
- **Stagnation dans les zones plates** : les mises à jour s'arrêtent presque complètement dans les zones où le gradient est proche de 0 (minima locaux).

Rappel : méthode de Newton

Comment résoudre les problème ?



Méthode de Newton

Pour une fonction $f(x)$ quadratiquement différentiable, considérons l'approximation de Taylor du second ordre de la fonction f objectif au point x_k .

$$f(x) = f(x_k) + \nabla f(x_k)^T (x - x_k) + \frac{1}{2} (x - x_k)^T \boxed{H_f(x_k)} (x - x_k) + o(\|x - x_k\|^2)$$

En négligeant les termes d'ordre supérieur $o((x - x_k)^2)$ et en minimisant la fonction, on obtient

$$x_{k+1} = x_k - \boxed{H_f(x_k)^{-1}} \nabla f(x_k)$$

Cette méthode itérative est connue sous le nom de **méthode de Newton classique**.

Preuve :

Supposons que le gradient de la fonction soit égal à 0

$$\nabla f(x) \approx \nabla f(x_k) + H_f(x_k)(x - x_k) = 0$$

La solution optimale en x_k est

$$x^* = x_k - H_f(x_k)^{-1} \nabla f(x_k)$$

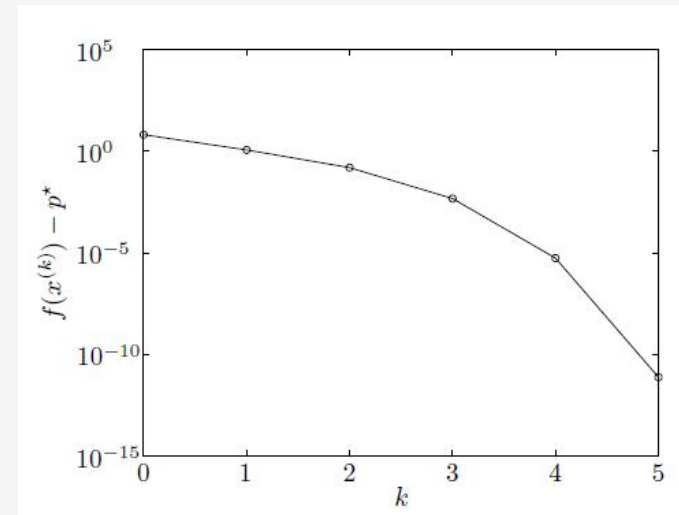
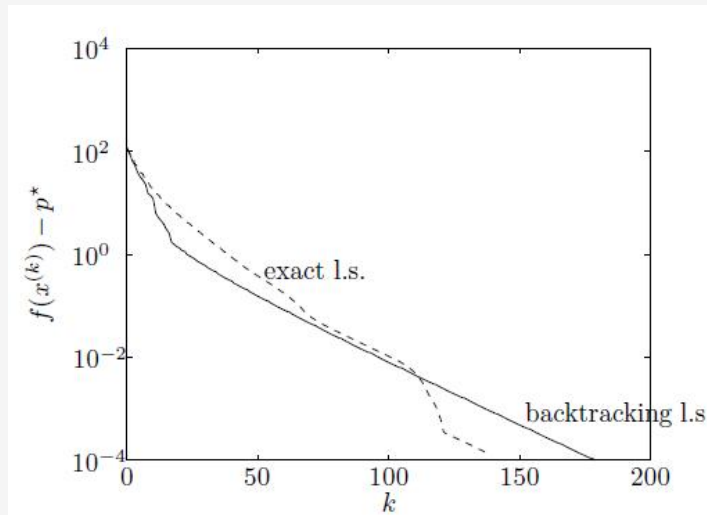
Méthode de Newton :
une méthode du second ordre à convergence plus rapide



Rappel : méthode de Newton

Descente de gradient : chaque étape se déplace dans la direction la plus raide, mais ne peut pas converger rapidement vers le point le plus bas.

Méthode de Newton : Introduction d'une méthode du second ordre à convergence plus rapide



La méthode de Newton converge très rapidement, mais elle est rarement utilisée en pratique en raison de sa complexité de calcul.



Peut-on trouver une méthode intermédiaire entre la descente de gradient et la méthode de Newton ?

Le rôle du momentum

La méthode de Newton utilise une matrice hessienne, et son essence réside dans la **modification du gradient**.



Fonction : $f(x_k) \approx f(x_{k-1}) + \nabla f(x_{k-1})(x_k - x_{k-1})$

De même, les variations du gradient peuvent être estimées à l'aide de plusieurs gradients.

Gradient : $\nabla f(x_k) \approx \nabla f(x_{k-1}) + H_f(x_{k-1})(x_k - x_{k-1})$

- Si la matrice hessienne est grande, le changement de gradient est significatif.
- Si la matrice hessienne est petite, le gradient reste quasiment constant.

Tenir compte de l'historique dans le processus de mise à jour

$$d_k = -\nabla f(x_k) \quad \rightarrow \quad d_k = -\nabla f(x_k) + \beta(x_k - x_{k-1})$$

Nouvelle formule mise à jour : $x_{k+1} = x_k - \alpha \nabla f(x_k) + \alpha \beta (x_k - x_{k-1})$

L'idée centrale du momentum : l'inertie

Momentum de Polyak

Introduire le momentum dans la formule de mise à jour

$$d_k = -\nabla f(x_k) + \beta(x_k - x_{k-1})$$
$$x_{k+1} = x_k + \alpha d_k$$

alors

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \alpha \beta (x_k - x_{k-1})$$

ou

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \mu (x_k - x_{k-1})$$

L'idée centrale du momentum : l'inertie

Momentum de Polyak

Introduire le momentum dans la formule de mise à jour

$$d_k = -\nabla f(x_k) + \beta(x_k - x_{k-1})$$

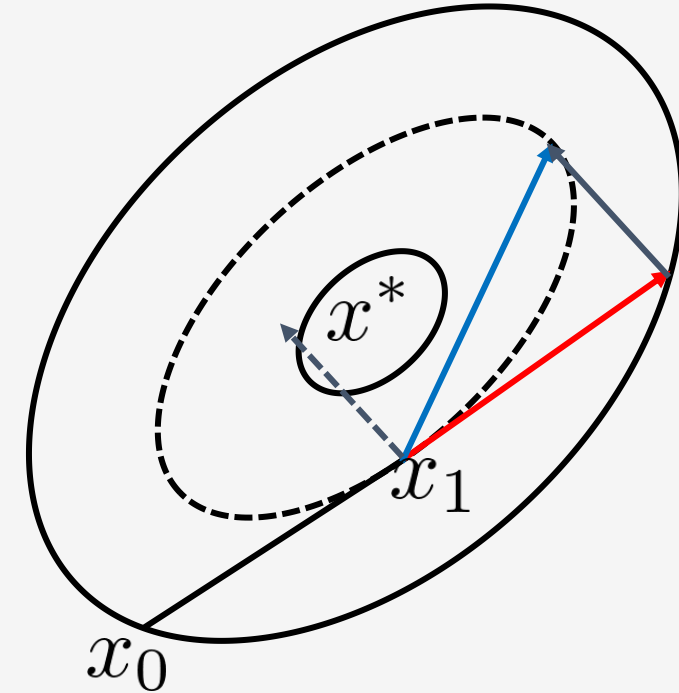
$$x_{k+1} = x_k + \alpha d_k$$

alors

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \alpha \beta (x_k - x_{k-1})$$

ou

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \mu (x_k - x_{k-1})$$



$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \mu (x_k - x_{k-1})$$

Descente de gradient vs Momentum

 How can I help you?

请帮我写一段代码，对比梯度下降法和动量梯度下降法（Momentum），并通过动图演示对比它们的收敛速度。

 DeepThink

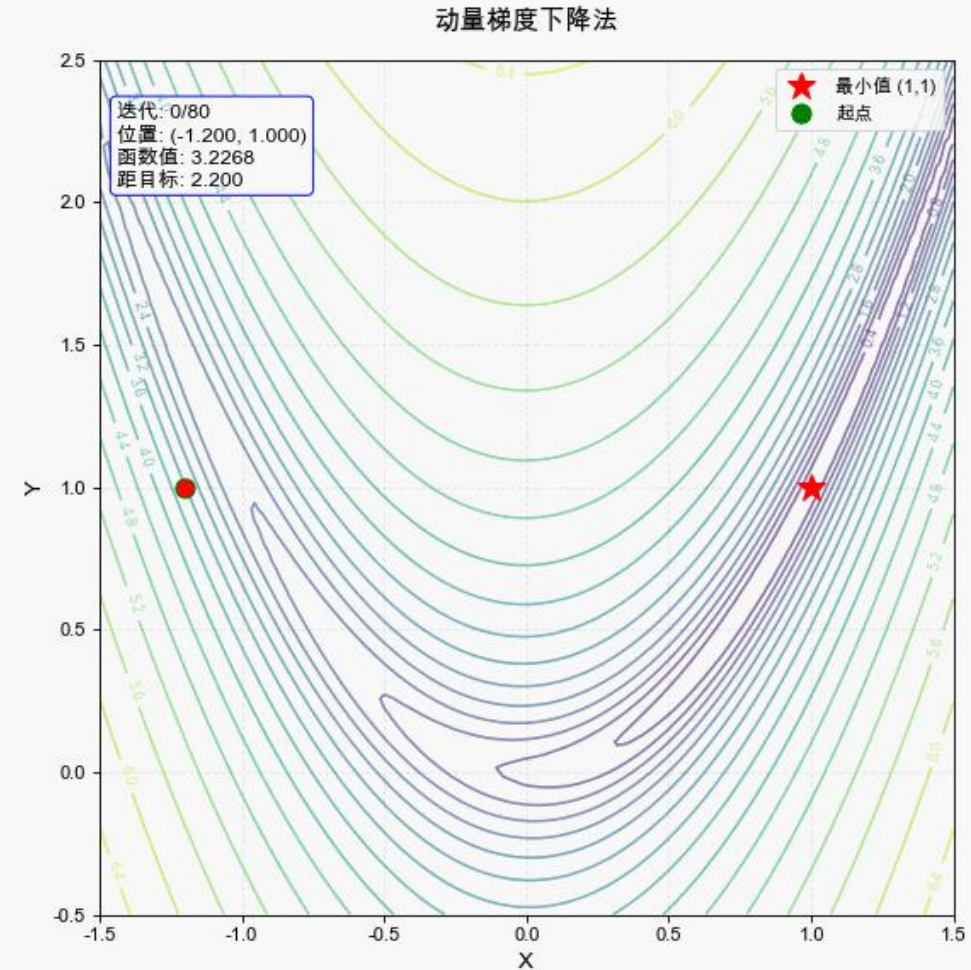
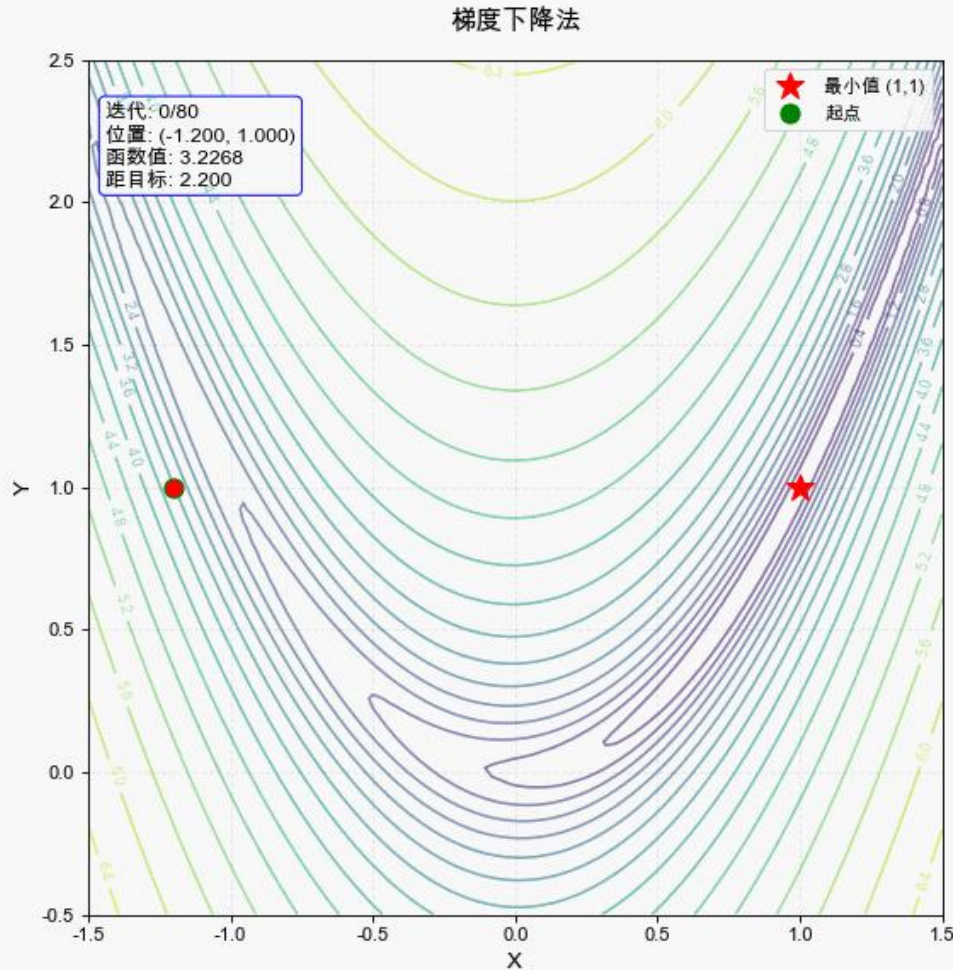
 Search





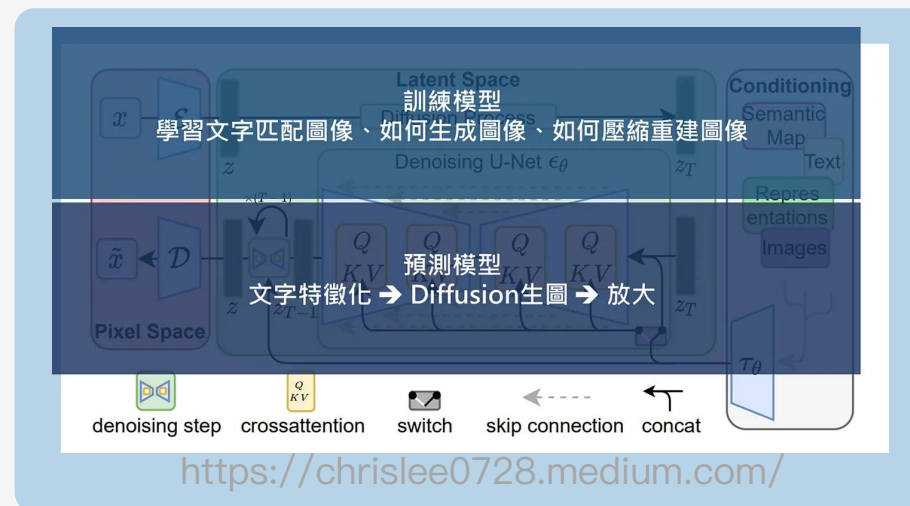
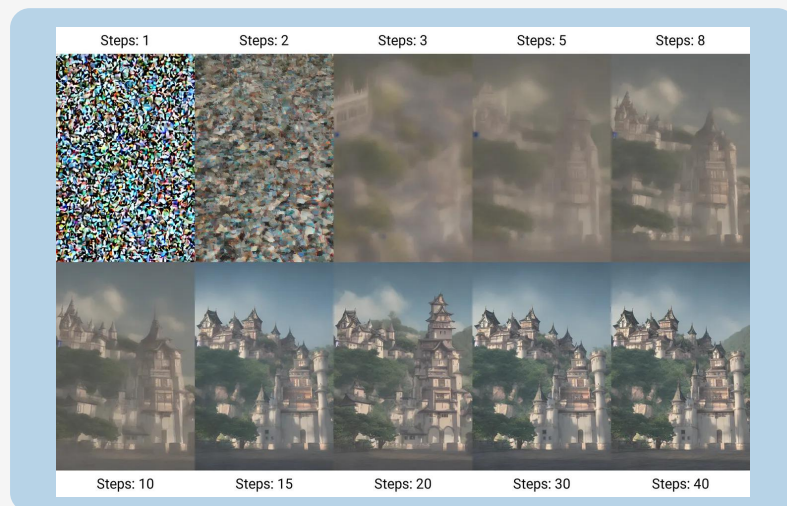
Descente de gradient vs Momentum

梯度下降法 vs 动量梯度下降法 收敛过程对比



La méthode Momentum permet de sortir des oscillations en accumulant de la quantité de mouvement, ce qui entraîne une vitesse de convergence plus rapide.

训练ChatGPT、Stable Diffusion这样的**千亿参数**大模型。



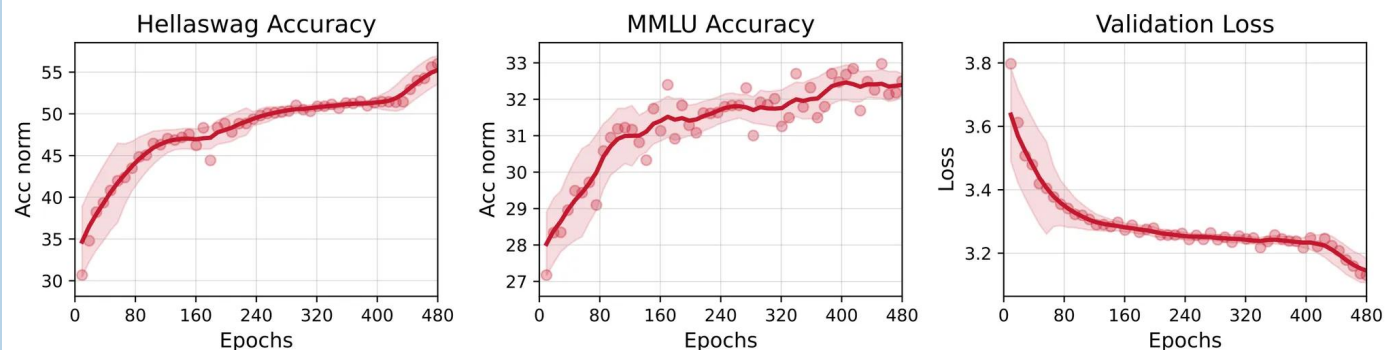
10亿参数DLM

训练480个轮次



每个参数都需要步长
损失函数复杂

1B Model, 1B Unique Tokens, 480 Epochs



Adam



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

每个参数都需要步长
损失函数复杂

Adam: 给每个参数配一个“智能步长”

Kingma et. al., 2014

变量	含义	作用
m (一阶矩)	梯度的指数移动平均	类似动量，加速收敛
v (二阶矩)	梯度平方的指数移动平均	自适应调整各参数的学习率

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} m_t$$

- θ : 神经网络的参数（权重和偏置）
- $\frac{\alpha}{\sqrt{v_t} + \epsilon}$: 各参数自己的学习率，且动态变化

arXiv:1412.6980v9 [cs.LG] 30 Jan 2017

Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma^{*}
University of Amsterdam
dpk1994@gmail.com

Jimmy Lei Ba^{*}
University of Toronto
jimmylpai.utoronto.ca

ABSTRACT

We introduce *Adam*, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which *Adam* was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that *Adam* works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of *Adam* based on the infinity norm.

1 INTRODUCTION

Stochastic gradient-based optimization is of core practical importance in many fields of science and engineering. Many problems in these fields can be cast as the optimization of some scalar parameterized objective function requiring maximization or minimization with respect to its parameters. If the function is differentiable w.r.t. its parameters, gradient descent is a relatively efficient optimization method, since the computation of first-order partial derivatives w.r.t. all the parameters is of the same computational complexity as just evaluating the function. Often, objective functions are stochastic. For example, many objective functions are composed of a sum of subfunctions evaluated at different subsamples of data; in this case optimization can be made more efficient by taking gradient steps w.r.t. individual subfunctions, i.e. stochastic gradient descent (SGD) or ascent. SGD proved itself as an efficient and effective optimization method that was central in many machine learning success stories, such as recent advances in deep learning (Deng et al., 2013; Krizhevsky et al., 2012; Hinton & Salakhutdinov, 2006; Hinton et al., 2012a; Graves et al., 2013). Objectives may also have other sources of noise than data subsampling, such as dropout (Hinton et al., 2012b) regularization. For all such noisy objectives, efficient stochastic optimization techniques are required. The focus of this paper is on the optimization of stochastic objectives with high-dimensional parameters spaces. In these cases, higher-order optimization methods are ill-suited, and discussion in this paper will be restricted to first-order methods.

We propose *Adam*, a method for efficient stochastic optimization that only requires first-order gradients with little memory requirement. The method computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients; the name *Adam* is derived from adaptive moment estimation. Our method is designed to combine the advantages of two recently popular methods: *AdaGrad* (Duchi et al., 2011), which works well with sparse gradients, and *RMSProp* (Tieleman & Hinton, 2012), which works well in on-line and non-stationary settings; important connections to these and other stochastic optimization methods are clarified in section 5. Some of *Adam*'s advantages are that the magnitudes of parameter updates are invariant to rescaling of the gradient, its stepsize is approximately bounded by the stepsize hyperparameter, it does not require a stationary objective, it works with sparse gradients, and it naturally performs a form of step size annealing.

^{*}Equal contribution. Author ordering determined by coin flip over a Google Hangout.

Published as a conference paper at ICLR 2015

basic optimization. See section 2 for details of computation. g_t^i indicates the elementwise machine learning problems are $\alpha = 0.001$, on vectors are element-wise. With β_1 and β_2

m : the moment estimates
 θ : parameters θ

f : objective at timestep t
 m : first moment estimate
 v : second raw moment estimate
 s : first moment estimate
 r : second raw moment estimate
 ϵ : (small)

series of its update rule. Section 3 explains in 4 provides a theoretical analysis of Adam's. Ifly, our method consistently outperforms other n in section 6. Overall, we show that Adam is mensional machine learning problems.

gorithm *Adam*. Let $f(\theta)$ be a noisy objective function w.r.t. parameters θ . We are in-
tion, $E[f(\theta)]$ w.r.t. its parameters θ . With
stochastic function at subsequent timesteps
diation at random subsamples (mini-batches)
With $g_t = \nabla_{\theta} f_t(\theta)$ we denote the gradient, i.e.
at timestep t .

of the gradient (m_t) and the squared gradient
the exponential decay rates of these moving
males of the 1st moment (the mean) and the
adient. However, these moving averages are
mates that are biased towards zero, especially
ray rates are small (i.e. the β s are close to 1),
asily counteracted, resulting in bias-corrected

nse of clarity, be improved upon by changing
use lines in the loop with the following lines:
 $\epsilon/(\sqrt{\epsilon_t} + \epsilon)$.

reful choice of signifiers. Assuming $\epsilon = 0$, the
 $\Delta_t = \alpha \cdot \beta_1 / \sqrt{\epsilon_t}$. The effective stepsize has
the case $(1 - \beta_1) > \sqrt{1 - \beta_2}$, and $\Delta_t \leq \alpha$

Kingma D P, Ba J. Adam: A method for stochastic optimization[J]. arXiv preprint arXiv:1412.6980, 2014.

每个参数都需要步长
损失函数复杂

Adam: 给每个参数配一个“智能步长”

Kingma et. al., 2014

梯度情况	实际学习率	效果
梯度一直很大	小	保守更新，防止震荡
梯度一直很小	大	激进更新，加速学习
梯度忽大忽小	中等	自适应平衡

arXiv:1412.6980v9 [cs.LG] 30 Jan 2017

Published as a conference paper at ICLR 2015

Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*

University of Amsterdam, OpenAI

dpk@openai.com

Jimmy Lei Ba*

University of Toronto

jimmy@psi.utoronto.ca

ABSTRACT

We introduce Adam, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal scaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which Adam was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that Adam works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of Adam based on the infinity norm.

1 INTRODUCTION

Stochastic gradient-based optimization is of core practical importance in many fields of science and engineering. Many problems in these fields can be cast as the optimization of some scalar parameterized objective function requiring maximization or minimization with respect to its parameters. If the function is differentiable w.r.t. its parameters, gradient descent is a relatively efficient optimization method, since the computation of first-order partial derivatives w.r.t. all the parameters is of the same computational complexity as just evaluating the function. Often, objective functions are stochastic. For example, many objective functions are composed of a sum of subfunctions evaluated at different subspaces of data; in this case optimization can be made more efficient by taking gradient steps w.r.t. individual subfunctions, i.e. stochastic gradient descent (SGD) or ascent. SGD proved itself as an efficient and effective optimization method that was central in many machine learning success stories, such as recent advances in deep learning (Deng et al., 2013; Krizhevsky et al., 2012; Hinton & Salakhutdinov, 2006; Hinton et al., 2012a; Graves et al., 2013). Objects may also have other sources of noise than data subsampling, such as dropout (Hinton et al., 2012b) regularization. For all such noisy objectives, efficient stochastic optimization techniques are required. The focus of this paper is on the optimization of stochastic objectives with high-dimensional parameters spaces. In these cases, higher-order optimization methods are ill-suited, and discussion in this paper will be restricted to first-order methods.

We propose Adam, a method for efficient stochastic optimization that only requires first-order gradients with little memory requirement. The method computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients; the name Adam is derived from adaptive moment estimation. Our method is designed to combine the advantages of two recently popular methods: AdaGrad (Duchi et al., 2011), which works well with sparse subgradients, and RMSProp (Tieleman & Hinton, 2012), which works well in on-line and non-stationary settings; important connections to these and other stochastic optimization methods are clarified in section 5. Some of Adam’s advantages are that the magnitudes of parameter updates are invariant to rescaling of the gradient, its stepsizes are approximately bounded by the stepsize hyperparameter, it does not require a stationary objective, it works with sparse gradients, and it naturally performs a form of on-line adaptive regularization.

basic optimization. See section 2 for details of optimization. g_t^2 indicates the element-wise machine learning problems are $\alpha = 0.01$ on vectors are element-wise. With β_1 and β_2

the moment estimates
rampers θ

objective at timestep t
first moment estimate
second raw moment estimate
st moment estimate
ad raw moment estimate
(ers)

series of its update rule. Section 3 explains in 4 provides a theoretical analysis of Adam’s. ally, our method consistently outperforms others in section 6. Overall, we show that Adam is a mensional machine learning problems.

gorithm Adam. Let $f(\theta)$ be a noisy objective function w.r.t. parameters θ . We are interested, $E[f(\theta)]$ w.r.t. its parameters θ . With stochastic function at subsequent timestep t evaluation at random subsamples (minibatches) with $g_t = \nabla_x f(\theta_t)$ we denote the gradient, i.e. at timestep t .

of the gradient (m_t) and the squared gradient (v_t) the exponential decay rates of these moving averages of the 1st moment (the mean) and the 2nd moment (the variance) respectively. However, these moving averages are estimates that are biased towards zero, especially at early rates are small (i.e. the β s are close to 1). Since of clatteriness, resulting in bias-correction.

ness of clarity, be improved upon by changing the lines in the loop with the following lines $\hat{m}_t \leftarrow (\beta_1 \hat{m}_t + (1 - \beta_1) g_t)$.

ref of choice of stepsizes. Assuming $\epsilon = 0$, the $\Delta_t = \alpha \cdot m_t / \sqrt{v_t}$. The effective stepsize has the case $(1 - \beta_1) > \sqrt{1 - \beta_2}$, and $\Delta_t \leq \alpha$.

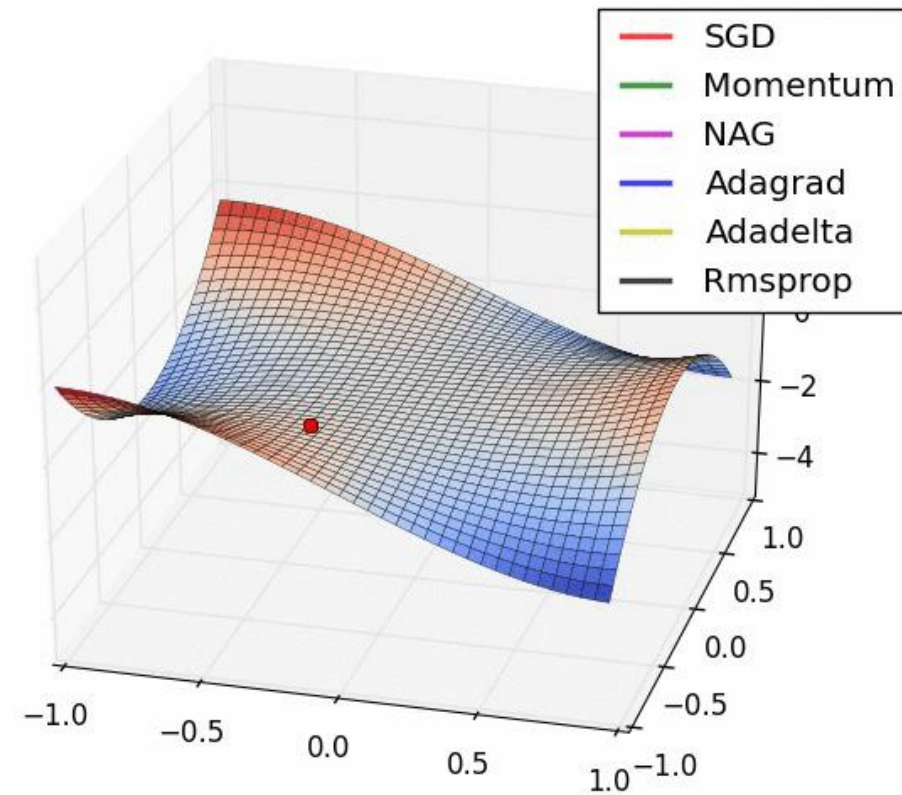
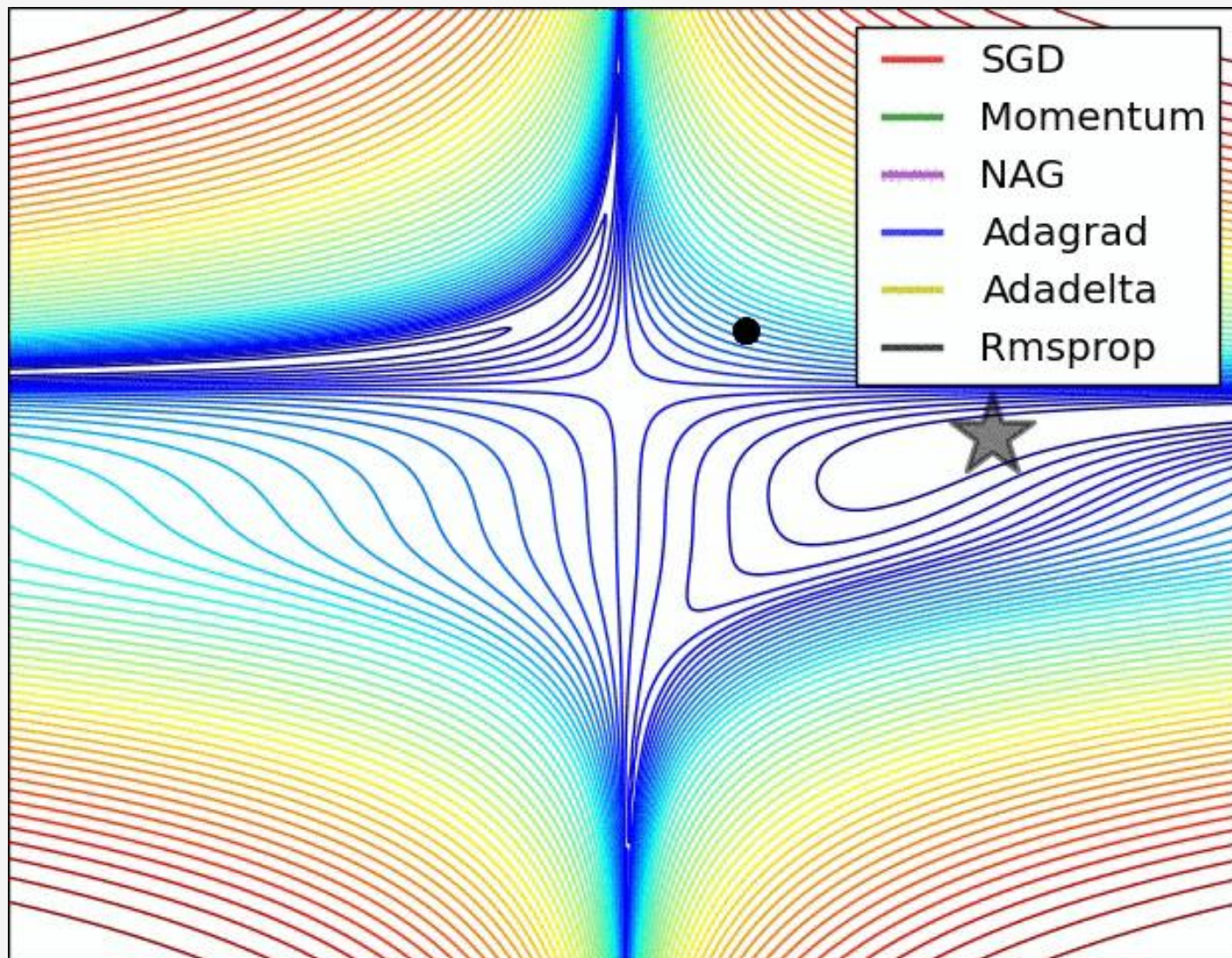
*Equal contribution. Author ordering determined by coin flip over a Google Hangout.



- L'algorithme Momentum est plus rapide.
- L'algorithme Momentum a plus de chances d'échapper aux minima locaux.

Comparaison du **Momentum (magenta)** et de la **descente de gradient (cyan)** sur des surfaces contenant un minimum global (piège de gauche) et un minimum local (piège de droite).

Taux d'apprentissage adaptatif

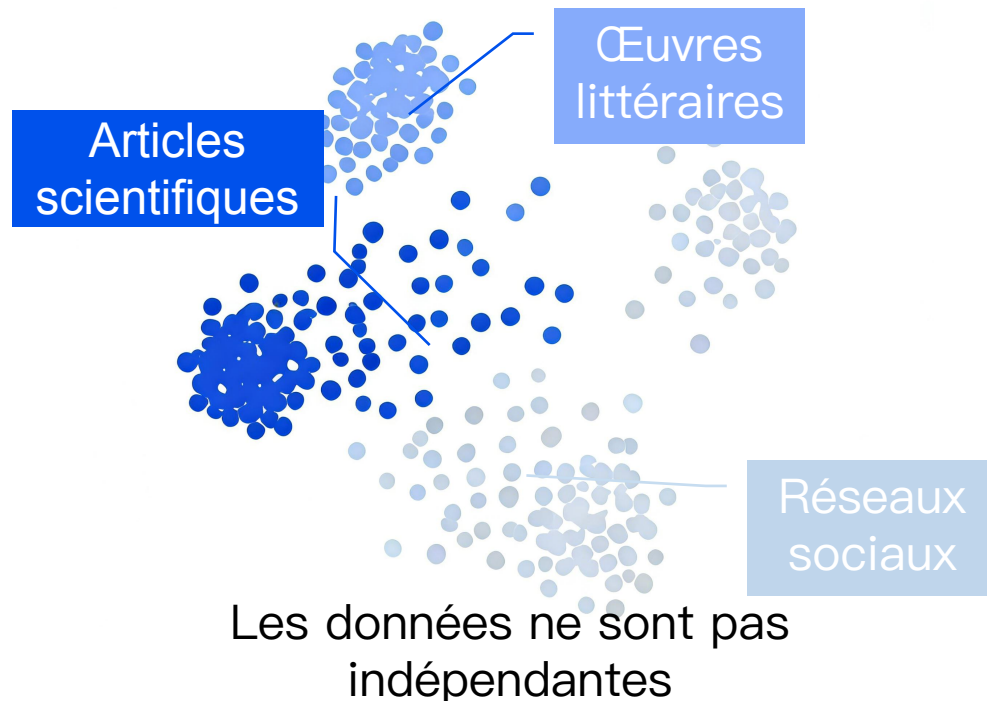


Annexe : Algorithme PISA



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

hétérogénéité des données



每个节点计算出的梯度区别明显，
直接使用模型会陷入严重的震荡，
甚至不收敛。

非精确求解

预条件机制

nature machine intelligence

Article

<https://doi.org/10.1038/s42256-026-01182-3>

Preconditioned inexact stochastic ADMM for deep models

Ouya Wang¹, Ziyao Luo¹, Yongxu Zhu² &

nature
machine
intelligence

els are usually trained with stochastic gradient
rithms, but these optimizers face inherent limitations,
rgence and stringent assumptions for convergence.
eterogeneity arising from distributed settings poses
ges to their theoretical and numerical performance.
i algorithm called PISA (preconditioned inexact
ing direction method of multipliers). Grounded in
l guarantees, the algorithm converges under the sole
chitz continuity of the gradient on a bounded region,
he need for other conditions commonly imposed
ods. This capability enables the proposed algorithm
nge of data heterogeneity effectively. Moreover,
hitecture enables scalable parallel computing and
reconditions, such as second-order information,
d orthogonalized momentum by Newton–Schulz
rating the last two preconditions in PISA yields two
ficient variants: SIS and NSIS. Comprehensive
iations for training or fine-tuning diverse deep models,
odels, large language models, reinforcement learning
adversarial networks and recurrent neural networks,
for numerical performance of SIS and NSIS
ious state-of-the-art optimizers.

(SGD) based optimizers due to their inherent limitations. In this sec-
tion, we review a subset of these methods, sufficient to motivate the
development of our proposed approach.

SGD-based learning algorithms
It is known that the plain SGD based algorithms, such as the original sto-
chastic approximation approaches^{1,2}, the parallelized SGD³ for machine
learning and the newly developed federated averaging (FedAvg⁴) and
localSGD (LocalSGD⁵) algorithms for federated learning, are frequently
prone to high sensitivity to poor conditioning⁶ and slow convergence

¹TP Laboratory, Department of EEE, Imperial College London,
at University, Nanjing, China. E-mail: shizhou@sjtu.edu.cn

234

334

A foundation
model for cardiac
health monitoring

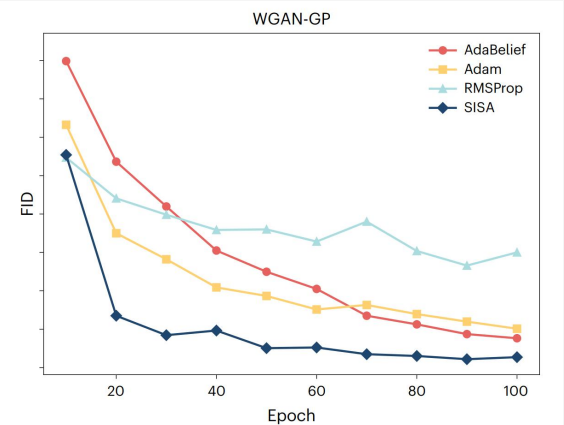
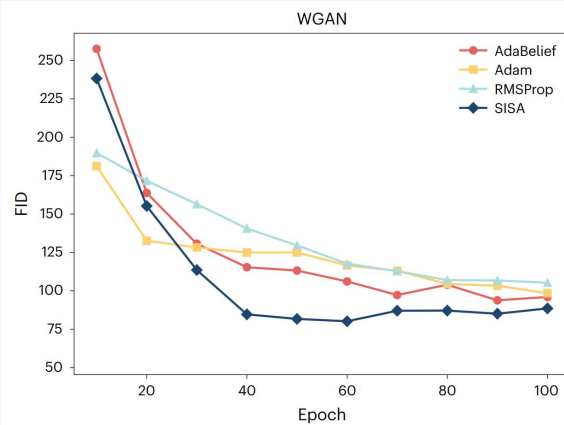
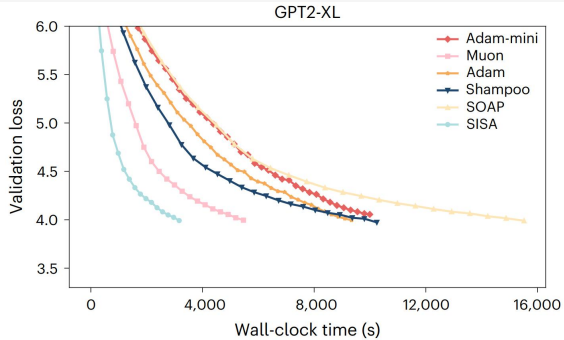
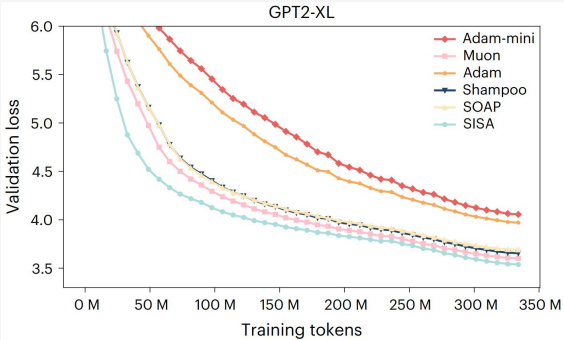
health monitoring
model for cardiac
A foundation

SJTU

Annexe : Algorithme PISA

PISA算法支持多种“预条件机制” (Preconditioning)

动量类型	Polyak动量	PISA (SISA)
本质	一阶动量	二阶动量
累积对象	梯度本身	梯度的平方
作用	平滑更新方向	自适应调节学习率
类比	速度	加速度的方差



PISA在大语言模型训练和图像处理方面都有明显的效率提升。

Méthode	Avantages	Inconvénients	Cas d'application
Descente de gradient	Calculs simples	Convergence lente sujette aux oscillations	Problèmes de grande taille
Momentum	Coût de calcul quasi inchangé	Nécessite toujours un réglage des paramètres	Une des méthodes les plus courantes en apprentissage profond
Newton	Convergence vite	Coûts de calcul et de stockage élevés	Optimisation convexe de petite taille, exigences de haute précision
quasi-Newton (L-BFGS)	Conserve certains information du 2 nd ordre	Sensible aux gradients stochastiques	Problèmes de taille moyenne, optimisation déterministe



上海交通大學

SHANGHAI JIAO TONG UNIVERSITY

Thank You

飲水思源 愛國榮校